

lib/remediation.ts

```
export type RemediationLanguage = "en" | "ar";

export type RemediationCategory =
  | "https-security"
  | "header-protection"
  | "infrastructure"
  | "reputation"
  | "tls";

export type RemediationSeverity = "critical" | "high" | "medium" | "low";
export type RemediationDifficulty = "easy" | "medium" | "hard";
export type RiskReduction = "high" | "medium" | "low";

export type LocalizedText = {
  en: string;
  ar: string;
};

export type RemediationEntry = {
  id: string;
  category: RemediationCategory;
  severity: RemediationSeverity;
  title: LocalizedText;
  explanation: {
    simple: LocalizedText;
    technical: LocalizedText;
  };
  businessImpact: LocalizedText;
  severityReason: LocalizedText;
  recommendation: LocalizedText;
  codeExamples: {
    vercel?: string;
    nginx?: string;
    apache?: string;
    cloudflare?: string;
  };
  difficulty: RemediationDifficulty;
  estimatedFixTime: LocalizedText;
  riskReduction: RiskReduction;
};

export type LocalizedRemediationEntry = Omit<
  RemediationEntry,
  | "title"
  | "explanation"
  | "businessImpact"
  | "severityReason"
  | "recommendation"
  | "estimatedFixTime"
> & {
  title: string;
  explanation: {
    simple: string;
    technical: string;
  };
  businessImpact: string;
  severityReason: string;
  recommendation: string;
  estimatedFixTime: string;
};

export type RemediationFindingId =
  | "missing-hsts"
  | "missing-csp"
  | "missing-x-frame-options"
  | "missing-x-content-type-options"
  | "missing-referrer-policy"
  | "missing-permissions-policy"
  | "weak-tls"
  | "expired-ssl"
  | "self-signed-ssl"
  | "ssl-expiring-soon"
  | "suspicious-redirect"
  | "redirect-loop"
  | "cross-domain-redirect"
  | "infrastructure-exposed"
  | "domain-too-new";

const commonHeaderExamples = {
  vercel: `{
    "headers": [
      {
        "source": "/(.*)",
        "headers": [
          { "key": "Strict-Transport-Security", "value": "max-age=31536000; includeSubDomains" },
          { "key": "X-Content-Type-Options", "value": "nosniff" },
          { "key": "Referrer-Policy", "value": "strict-origin-when-cross-origin" },
          { "key": "Permissions-Policy", "value": "camera=(), microphone=(), geolocation=()" },
          { "key": "Content-Security-Policy", "value": "default-src 'self'; frame-ancestors 'self'; object-src 'none'; base-uri 'self'" }
        ]
      }
    ]
  }`,
  nginx: `add_header Strict-Transport-Security "max-age=31536000; includeSubDomains" always;
add_header X-Content-Type-Options "nosniff" always;
add_header Referrer-Policy "strict-origin-when-cross-origin" always;
add_header Permissions-Policy "camera=(), microphone=(), geolocation=()" always;
add_header Content-Security-Policy "default-src 'self'; frame-ancestors 'self'; object-src 'none'; base-uri 'self'" always;`,
  apache: `Header always set Strict-Transport-Security "max-age=31536000; includeSubDomains"
Header always set X-Content-Type-Options "nosniff"
Header always set Referrer-Policy "strict-origin-when-cross-origin"
Header always set Permissions-Policy "camera=(), microphone=(), geolocation=()"
Header always set Content-Security-Policy "default-src 'self'; frame-ancestors 'self'; object-src 'none';
base-uri 'self'`;
};

export const remediationMap = {
  "missing-hsts": {
    id: "missing-hsts",
    category: "https-security",
    severity: "high",
    title: {
      en: "Missing HTTP Strict Transport Security",
      ar: "الخطأ في إعدادات HSTS",
    },
  },
}
```

```
,
explanation: {
  simple: {
    en: "The site does not tell browsers to always use HTTPS for future visits.",
    ar: "المتصفح لا يخبر المستخدمين باستخدام HTTPS لزياراتهم المستقبلية.",
  },
  technical: {
    en: "Strict-Transport-Security is absent, so browsers may still attempt insecure HTTP connections before upgrading to HTTPS.",
    ar: "غياب Strict-Transport-Security يعني أن المتصفحات قد تحاول الاتصال بالمتصفح عبر HTTP غير الآمن قبل الترقية إلى HTTPS.",
  },
},
businessImpact: {
  en: "Users on public or hostile networks may be exposed to interception attempts before the secure connection is enforced.",
  ar: "المستخدمون على شبكات عامة أو غير آمنة قد يتعرضون لمحاولات اعتراض البيانات قبل تفعيل الاتصال الآمن.",
},
severityReason: {
  en: "This is high severity because it weakens HTTPS enforcement and can enable downgrade-style attacks.",
  ar: "هذه مشكلة خطيرة لأنها تضعف تطبيق HTTPS ويمكن أن تتيح هجمات الترقية-النزول.",
},
recommendation: {
  en: "Enable HSTS with a long max-age after confirming that every page and subdomain works correctly over HTTPS.",
  ar: "تفعيل HSTS مع max-age طويل بعد التأكد من أن كل صفحة ودومين يعملان بشكل صحيح عبر HTTPS.",
},
codeExamples: {
  vercel: commonHeaderExamples.vercel,
  nginx: `add_header Strict-Transport-Security "max-age=31536000; includeSubDomains" always;`,
  apache: `Header always set Strict-Transport-Security "max-age=31536000; includeSubDomains"`,
},
difficulty: "easy",
estimatedFixTime: {
  en: "15 minutes",
  ar: "15 دقائق",
},
riskReduction: "high",
},
"missing-csp": {
  id: "missing-csp",
  category: "header-protection",
  severity: "high",
  title: {
    en: "Missing Content Security Policy",
    ar: "السياسة الأمنية للمحتوى مفقودة",
  },
  explanation: {
    simple: {
      en: "The browser has fewer rules to block unsafe scripts, frames, and external resources.",
      ar: "المتصفح لديه قواعد أقل لمنع scripts غير آمنة، إطارات، وموارد خارجية.",
    },
    technical: {
      en: "Content-Security-Policy is absent, reducing browser-side protection against script injection, clickjacking, and untrusted resource loading.",
      ar: "غياب Content-Security-Policy يقلل الحماية على جانب المتصفح ضد حقن scripts، كليكجacking، وتحميل الموارد غير موثوقة.",
    },
  },
  businessImpact: {
    en: "A successful injection issue could lead to account abuse, session theft, or damaged user trust.",
    ar: "مشكلة حقن ناجحة قد تؤدي إلى إساءة استخدام الحساب، سرقة الجلسات، أو فقدان ثقة المستخدمين.",
  },
  severityReason: {
    en: "This is high severity because CSP is a major browser control that limits the impact of several common web attacks.",
    ar: "هذه مشكلة خطيرة لأن CSP هي سيطرة رئيسية في المتصفح تحد من تأثير العديد من هجمات الويب الشائعة.",
  },
  recommendation: {
    en: "Start with a restrictive policy for default sources, scripts, frames, objects, and base URI, then expand only for trusted services.",
    ar: "ابدأ بسياسة صارمة للمصادر الافتراضية، scripts، إطارات، كائنات، وURI الأساسية، ثم توسع فقط للخدمات موثوقة.",
  },
  codeExamples: {
    vercel: commonHeaderExamples.vercel,
    nginx: `add_header Content-Security-Policy "default-src 'self'; frame-ancestors 'self'; object-src 'none'; base-uri 'self'" always;`,
    apache: `Header always set Content-Security-Policy "default-src 'self'; frame-ancestors 'self'; object-src 'none'; base-uri 'self'"`,
  },
  difficulty: "medium",
  estimatedFixTime: {
    en: "1-2 hours",
    ar: "1-2 ساعات",
  },
  riskReduction: "high",
},
"missing-x-frame-options": {
  id: "missing-x-frame-options",
  category: "header-protection",
  severity: "medium",
  title: {
    en: "Missing Frame Protection",
    ar: "حماية الإطار مفقودة",
  },
  explanation: {
    simple: {
      en: "Other sites may be able to embed your pages inside frames.",
      ar: "مواقع أخرى قد تتمكن من تضمين صفحاتك داخل إطارات.",
    },
    technical: {
      en: "X-Frame-Options is absent, and without a CSP frame-ancestors rule the page may be exposed to clickjacking.",
      ar: "غياب X-Frame-Options، وبدون قاعدة frame-ancestors في CSP، قد تتعرض الصفحة لهجمات كليكجacking.",
    },
  },
  businessImpact: {
    en: "Users can be tricked into clicking hidden login, payment, or account actions on a malicious page.",
    ar: "يمكنخد المستخدمين بالنقر على أزرار مخفية (تسجيل، دفع، أو إجراءات الحساب) على صفحة خبيثة.",
  },
  severityReason: {
    en: "This is medium severity because it mainly affects sensitive interactive pages and user actions.",
    ar: "هذه مشكلة متوسطة الخطورة لأنها تؤثر بشكل رئيسي على صفحات تفاعلية حساسة وأفعال المستخدمين.",
  },
}
```

```

    ar: "???? ?????? ?????? ?????? ???? ?????? ??? ?????? ?????????? ?????? ????????? ?????????.",
  },
  recommendation: {
    en: "Set X-Frame-Options to SAMEORIGIN or DENY, and prefer CSP frame-ancestors for modern control.",
    ar: "???? X-Frame-Options ??? SAMEORIGIN ?? DENY? ????? ?????? frame-ancestors ??? CSP ?????? ??????",
  },
  codeExamples: {
    vercel: `{
"headers": [
  {
    "source": "/(.*)",
    "headers": [
      { "key": "X-Frame-Options", "value": "SAMEORIGIN" }
    ]
  }
]
}`,
    nginx: `add_header X-Frame-Options "SAMEORIGIN" always;`,
    apache: `Header always set X-Frame-Options "SAMEORIGIN";`,
  },
  difficulty: "easy",
  estimatedFixTime: {
    en: "10 minutes",
    ar: "10 ?????",
  },
  riskReduction: "medium",
},
"missing-x-content-type-options": {
  id: "missing-x-content-type-options",
  category: "header-protection",
  severity: "medium",
  title: {
    en: "Missing MIME Sniffing Protection",
    ar: "???? ????? MIME Sniffing",
  },
  explanation: {
    simple: {
      en: "Browsers may try to guess file types instead of trusting the declared content type.",
      ar: "?? ????? ?????? ????? ?? ?????? ????? ?? ?????????? ?? ?? ?????? ??????",
    },
    technical: {
      en: "X-Content-Type-Options is absent, allowing MIME sniffing behavior that can turn misclassified files into executable content.",
      ar: "??? X-Content-Type-Options ??? ?????? ?? ?????? ????? MIME sniffing ??? ?? ?????? ?????? ?????? ?? ?? ?????? ?????? ?????? ??????",
    },
  },
  businessImpact: {
    en: "A content handling mistake could become a security issue if uploaded or downloaded files are interpreted unsafely.",
    ar: "?? ?????? ?? ?? ?????? ?? ?????? ?? ?????? ?????? ?? ?????? ?????? ?? ?????? ?????? ?? ??????",
  },
  severityReason: {
    en: "This is medium severity because the fix is simple and prevents a known class of browser misinterpretation attacks.",
    ar: "???? ?????? ?????? ?????? ?????? ?????? ?????? ?????? ?????? ?? ?????? ?? ?????? ?????? ?????? ??????",
  },
  recommendation: {
    en: "Set X-Content-Type-Options to nosniff on all responses.",
    ar: "???? X-Content-Type-Options ??? nosniff ?? ?????? ??????????",
  },
  codeExamples: {
    vercel: `{
"headers": [
  {
    "source": "/(.*)",
    "headers": [
      { "key": "X-Content-Type-Options", "value": "nosniff" }
    ]
  }
]
}`,
    nginx: `add_header X-Content-Type-Options "nosniff" always;`,
    apache: `Header always set X-Content-Type-Options "nosniff";`,
  },
  difficulty: "easy",
  estimatedFixTime: {
    en: "5 minutes",
    ar: "5 ?????",
  },
  riskReduction: "medium",
},
"missing-referrer-policy": {
  id: "missing-referrer-policy",
  category: "header-protection",
  severity: "low",
  title: {
    en: "Missing Referrer Policy",
    ar: "???? ????? Referrer",
  },
  explanation: {
    simple: {
      en: "Browsers may share more URL information with other sites than necessary.",
      ar: "?? ?????? ?????? ?????? ?? ?????? ?? ?????? ?????? ?? ?? ??????",
    },
    technical: {
      en: "Referrer-Policy is absent, so full paths, query strings, or campaign tokens may be sent during navigation.",
      ar: "??? Referrer-Policy ??? ?????? ?? ?? ?????? ?????????? ?????? ?? ?????? ?????? ?? ?????? ??????",
    },
  },
  businessImpact: {
    en: "Private paths or tracking tokens may leak to third parties, creating privacy and compliance concerns.",
    ar: "?? ?????? ?????? ??? ?? ??? ??? ?? ?????? ?????? ?? ??? ?????? ?????? ??????",
  },
  severityReason: {
    en: "This is low severity because it usually exposes metadata rather than direct access, but it still improves privacy posture.",
    ar: "???? ?????? ?????? ?????? ?????? ?????? ?????? ?????? ?????? ?? ?????? ?????????? ?????? ?? ??????",
  },
  recommendation: {

```

```
en: "Set Referrer-Policy to strict-origin-when-cross-origin or stricter if the application can support it.",
ar: "???? Referrer-Policy ??? strict-origin-when-cross-origin ?? ????? ???? ?????? ??? ??? ??????
???? ????.",
},
codeExamples: {
vercel: `{
"headers": [
{
"source": "/(.*)",
"headers": [
{ "key": "Referrer-Policy", "value": "strict-origin-when-cross-origin" }
]
}
]
}
},
nginx: `add_header Referrer-Policy "strict-origin-when-cross-origin" always;`,
apache: `Header always set Referrer-Policy "strict-origin-when-cross-origin";`,
},
difficulty: "easy",
estimatedFixTime: {
en: "5 minutes",
ar: "5 ?????",
},
riskReduction: "low",
},
"missing-permissions-policy": {
id: "missing-permissions-policy",
category: "header-protection",
severity: "low",
title: {
en: "Missing Permissions Policy",
ar: "???? Permissions Policy",
},
explanation: {
simple: {
en: "Browser features such as camera, microphone, and location are not restricted by policy.",
ar: "???? ?????? ??? ?????? ?????????? ?????? ??? ?????? ?????? ??????",
},
technical: {
en: "Permissions-Policy is absent, so unused browser capabilities are not explicitly disabled for the origin.",
ar: "??? Permissions-Policy ??? ?????? ??? ?? ??? ?????? ?????? ?????? ??? ?????????? ?????? ?????
?????",
},
},
businessImpact: {
en: "The site may appear less privacy-conscious, especially for products handling user accounts or sensitive workflows.",
ar: "?? ??? ?????? ??? ?????????? ?????????? ?????? ??? ?????? ?? ?????? ?????????? ?? ?????? ?????.",
},
severityReason: {
en: "This is low severity because it is a hardening control, but it reduces unnecessary browser attack surface.",
ar: "???? ?????? ?????? ?????? ?????? ?????? ?????? ?????? ?????? ??? ?????? ?????? ??????",
},
recommendation: {
en: "Disable browser capabilities that the site does not need, then allow features only where required.",
ar: "???? ?????? ?????? ??? ?? ?????? ?????? ?? ??? ?????????? ??? ??? ??????",
},
codeExamples: {
vercel: `{
"headers": [
{
"source": "/(.*)",
"headers": [
{ "key": "Permissions-Policy", "value": "camera=(), microphone=(), geolocation=()" }
]
}
]
}
},
nginx: `add_header Permissions-Policy "camera=(), microphone=(), geolocation=()" always;`,
apache: `Header always set Permissions-Policy "camera=(), microphone=(), geolocation=()";`,
},
difficulty: "easy",
estimatedFixTime: {
en: "10 minutes",
ar: "10 ?????",
},
riskReduction: "low",
},
"weak-tls": {
id: "weak-tls",
category: "tls",
severity: "high",
title: {
en: "Weak TLS Configuration",
ar: "?????? TLS ?????",
},
explanation: {
simple: {
en: "The encrypted connection may support older protocols or weak cipher choices.",
ar: "?? ??? ?????? ?????? ?????????? ?????? ?? ?????? ?????? ??????",
},
technical: {
en: "Deprecated TLS versions or weak ciphers increase exposure to downgrade, compatibility, and cryptographic attacks.",
ar: "?????? TLS ?????? ?? ?????????? ?????? ?????? ??? ?????? ?????? ??? ?????? ?????? ?????
?? ??????????",
},
},
businessImpact: {
en: "Customers may lose confidence if the site appears to use outdated transport security.",
ar: "?? ??? ?????? ?????? ??? ?? ?? ?????? ?????? ?????? ??? ?????? ??????",
},
severityReason: {
en: "This is high severity because TLS protects credentials, sessions, and sensitive user traffic in transit.",
ar: "???? ?????? ?????? ?????? ??? TLS ??? ?????? ?????? ?????????? ?????? ?????????? ?????? ???
??.",
},
recommendation: {
en: "Disable TLS 1.0 and 1.1, enable TLS 1.2 or TLS 1.3, and use modern cipher suites.",
ar: "???? TLS 1.0 ? TLS 1.1? ?????? TLS 1.2 ?? TLS 1.3 ?? ??? ?????? ??????",
},
codeExamples: {
nginx: `ssl_protocols TLSv1.2 TLSv1.3;`
}
```

[illegible]

```
businessImpact: {
  en: "A missed renewal can suddenly create browser warnings, support tickets, and lost conversions.",
  ar: "?? ????? ??????? ?? ??????? ??????? ?? ??????? ??????? ?? ??????? ????????",
},
severityReason: {
  en: "This is medium severity because the site is still trusted today, but the failure window is approaching.",
  ar: "???? ??????? ??????? ??????? ?? ??????? ?? ?? ??????? ??????? ?? ??????? ??????? ????????",
},
recommendation: {
  en: "Renew the certificate early and verify that automated renewal and expiry notifications are working.",
  ar: "???? ??????? ??????? ??????? ?? ?? ??????? ??????? ??????? ??????? ??????? ????????",
},
codeExamples: {},
difficulty: "easy",
estimatedFixTime: {
  en: "15 minutes",
  ar: "15 ??????",
},
riskReduction: "medium",
},
"suspicious-redirect": {
  id: "suspicious-redirect",
  category: "infrastructure",
  severity: "high",
  title: {
    en: "Suspicious Redirect Behavior",
    ar: "???? ????? ??????",
  },
  explanation: {
    simple: {
      en: "The site redirects users in a way that may lead to unexpected or untrusted destinations.",
      ar: "?????? ????? ?????????? ?????? ?? ????? ?? ????? ?? ????? ?? ????? ?? ??????",
    },
    technical: {
      en: "Redirect behavior appears unusual, excessive, or aligned with patterns commonly seen in phishing or traffic laundering flows.",
      ar: "???? ??????? ?????? ?? ?????? ?? ??????? ?????? ????? ?? ?????? ?? ?????? ?????????? ?? ?????? ?? ??????",
    },
  },
  businessImpact: {
    en: "Users may lose trust if they start on your domain and end up somewhere unexpected.",
    ar: "?? ??? ?????????? ?????? ?? ?????? ?? ?????? ??????? ?? ??? ?? ??????",
  },
  severityReason: {
    en: "This is high severity because redirect abuse can quickly turn a trusted entry point into a phishing path.",
    ar: "???? ??????? ?????? ??????? ?? ?????? ??????? ?????????? ?? ?????? ?? ?????? ?? ?????? ?? ??????",
  },
  recommendation: {
    en: "Review redirect rules, remove unexpected destinations, and allow only approved domains owned or trusted by the organization.",
    ar: "???? ?????? ?????????? ?????? ??????? ?????? ?? ?????????? ?????????? ?????????? ?? ?????? ?? ??????",
  },
  codeExamples: {
    nginx: `return 301 https://example.com$request_uri;`,
    apache: `Redirect permanent / https://example.com/`,
  },
  difficulty: "medium",
  estimatedFixTime: {
    en: "30 minutes",
    ar: "30 ??????",
  },
  riskReduction: "high",
},
"redirect-loop": {
  id: "redirect-loop",
  category: "infrastructure",
  severity: "medium",
  title: {
    en: "Redirect Loop",
    ar: "???? ????? ??????",
  },
  explanation: {
    simple: {
      en: "The site may send visitors between URLs repeatedly instead of reaching a final page.",
      ar: "?? ??? ?????? ?????????? ?? ?????? ?? ?????? ?????? ?? ?????? ?? ?????? ?? ?????? ??????",
    },
    technical: {
      en: "Redirect rules appear to cycle between locations, which can prevent clients and scanners from reaching a stable final URL.",
      ar: "????? ??????? ??? ?????? ??? ?? ?????? ??????? ?? ?????? ?????????? ?????????? ?? ?????? ?? ???? ?????? ??????",
    },
  },
  businessImpact: {
    en: "Visitors may be unable to access the site, creating failed sessions and avoidable support issues.",
    ar: "?? ?? ?????? ?????? ?? ?????? ?? ??????? ?? ?????? ?????? ?????? ?????? ?? ?????? ??????",
  },
  severityReason: {
    en: "This is medium severity because it mainly affects availability and user trust rather than direct data exposure.",
    ar: "???? ?????????? ?????? ??????? ?????? ?????? ?????? ?? ?????? ?????? ?????? ?????? ?? ?????? ??????",
  },
  recommendation: {
    en: "Normalize HTTP-to-HTTPS and canonical-domain rules so every request reaches one stable final destination.",
    ar: "????? ?????? ?????????? ?? HTTP ?? HTTPS ??????? ??????? ?????? ?? ?? ?? ?? ?? ?????? ??????? ???",
  },
  codeExamples: {
    nginx: `server {
  listen 80;
  server_name example.com www.example.com;
  return 301 https://example.com$request_uri;
}`,
    apache: `RewriteEngine On
RewriteCond %{HTTPS} !=on
RewriteRule ^ https://example.com%{REQUEST_URI} [R=301,L]`,
  },
  difficulty: "medium",
  estimatedFixTime: {
    en: "20 minutes",
  },
}
```

```

    ar: "20 ?????",
  },
  riskReduction: "medium",
},
"cross-domain-redirect": {
  id: "cross-domain-redirect",
  category: "infrastructure",
  severity: "medium",
  title: {
    en: "Cross-Domain Redirect",
    ar: "????? ??? ????? ?????",
  },
  explanation: {
    simple: {
      en: "The site sends users from the original domain to another domain.",
      ar: "??????? ??? ?????????? ?? ?????? ?????? ??? ????? ???.",
    },
    technical: {
      en: "The redirect chain leaves the original host, which expands the trust boundary and may weaken control over the user journey.",
      ar: "????? ?????????? ??? ?? ?????? ??????? ??? ??? ??? ?????? ??? ??? ?????? ?? ??? ??????",
    },
  },
  businessImpact: {
    en: "Users may be confused or concerned when a trusted site sends them to a different domain.",
    ar: "?? ????? ?????????? ?????????? ?? ?????? ?????? ?????? ??? ?????? ?? ?????? ??????",
  },
  severityReason: {
    en: "This is medium severity because cross-domain movement can be legitimate, but it increases trust and monitoring requirements.",
    ar: "???? ?????? ?????? ?????? ??? ?????? ??? ?????????? ?? ?????? ?????????? ??? ?????? ?????? ?????? ??????????",
  },
  recommendation: {
    en: "Keep cross-domain redirects limited to approved destinations and document the owner and purpose of each destination.",
    ar: "???? ?????????? ??? ?????????? ?????? ??? ?????? ?????????? ?????? ??? ?? ??? ?????? ??????????",
  },
  codeExamples: {
    cloudflare: `(http.request.uri.path eq "/old-path")`,
  },
  difficulty: "medium",
  estimatedFixTime: {
    en: "20 minutes",
    ar: "20 ?????",
  },
  riskReduction: "medium",
},
"infrastructure-exposed": {
  id: "infrastructure-exposed",
  category: "infrastructure",
  severity: "low",
  title: {
    en: "Infrastructure Metadata Exposed",
    ar: "??? ?????? ?????? ?????????",
  },
  explanation: {
    simple: {
      en: "The site reveals details about servers, frameworks, or hosting technology.",
      ar: "??????? ??? ?????? ?? ?????? ?? ?????? ?????????? ?? ?????? ??????????",
    },
    technical: {
      en: "Server, framework, or platform headers expose implementation details that can help attackers prioritize known weaknesses.",
      ar: "????? Server ?? Framework ?? ?????? ??? ?????? ?????? ?????????? ?? ?????? ?????????? ?? ?????? ??????????",
    },
  },
  businessImpact: {
    en: "Exposed stack details can make targeted attacks easier and reduce the perceived maturity of the security posture.",
    ar: "??? ?????? ?????? ?? ??? ?????? ?????? ??? ?????? ?????????? ??? ?????? ??????",
  },
  severityReason: {
    en: "This is low severity because metadata alone is not a breach, but it improves attacker reconnaissance.",
    ar: "???? ?????? ?????? ?????? ??? ?????????? ?????? ?????? ?????? ?????? ?????? ??? ?????? ??????",
  },
  recommendation: {
    en: "Remove unnecessary server and framework headers, and keep operational diagnostics internal.",
    ar: "???? ??? Server ?Framework ??? ?????????? ?????? ?????? ?????? ?????????? ??????",
  },
  codeExamples: {
    nginx: `server_tokens off;`,
    apache: `ServerTokens Prod
ServerSignature Off`,
  },
  difficulty: "easy",
  estimatedFixTime: {
    en: "10 minutes",
    ar: "10 ?????",
  },
  riskReduction: "low",
},
"domain-too-new": {
  id: "domain-too-new",
  category: "reputation",
  severity: "medium",
  title: {
    en: "Recently Registered Domain",
    ar: "???? ??? ??????",
  },
  explanation: {
    simple: {
      en: "The domain appears new, which can reduce trust until reputation is established.",
      ar: "???? ?? ?????? ?????? ??? ?? ??? ?????? ??? ?????? ?? ??? ??????",
    },
    technical: {
      en: "Newly registered domains are frequently used in short-lived phishing and abuse campaigns before reputation systems classify them.",
      ar: "????????? ?????????? ?????????? ?????? ?? ?????? ??? ?????? ?????? ?????? ??? ?? ?????? ?????? ??????",
    },
  },
  businessImpact: {
    en: "Customers, partners, and security tools may treat a new domain with extra caution.",
    ar: "?? ?????? ?????????? ?????????? ?????? ?????? ??? ?????? ?????? ??? ??????",
  },
}

```

```

    },
    severityReason: {
      en: "This is medium severity because age is not proof of abuse, but it is a meaningful trust signal."
    },
    ar: "???? ?????? ?????? ?????? ??? ?????? ?????? ?? ?????? ?????? ?????? ??? ??? ???.",
  },
  recommendation: {
    en: "Use stronger verification, monitoring, email authentication, and brand-trust controls while the domain builds reputation.",
    ar: "????? ?????? ?????? ??? ?????? ??? ?????? ??? ?????? ?????? ?????? ??? ??? ??????.",
  },
  codeExamples: {
    cloudFlare: `(ip.geoip.asnum in {13335})`,
  },
  difficulty: "medium",
  estimatedFixTime: {
    en: "1 hour",
    ar: "???? ??????",
  },
  riskReduction: "medium",
},
} satisfies Record<RemediationFindingId, RemediationEntry>;

export function getRemediationById(id: string): RemediationEntry | undefined {
  return remediationMap[id as RemediationFindingId];
}

export function getLocalizedRemediation(
  id: string,
  lang: RemediationLanguage,
): LocalizedRemediationEntry | undefined {
  const remediation = getRemediationById(id);

  if (!remediation) {
    return undefined;
  }

  return {
    id: remediation.id,
    category: remediation.category,
    severity: remediation.severity,
    title: remediation.title[lang],
    explanation: {
      simple: remediation.explanation.simple[lang],
      technical: remediation.explanation.technical[lang],
    },
    businessImpact: remediation.businessImpact[lang],
    severityReason: remediation.severityReason[lang],
    recommendation: remediation.recommendation[lang],
    codeExamples: remediation.codeExamples,
    difficulty: remediation.difficulty,
    estimatedFixTime: remediation.estimatedFixTime[lang],
    riskReduction: remediation.riskReduction,
  };
}

```


lib/pdf.ts

```
import type { Language, Translations } from "@lib/i18n";
import { getRemediationById } from "@lib/remediation";
import type { RemediationEntry } from "@lib/remediation";
import type { AIExplanation, Finding, ScanResult } from "@lib/types";

type PdfOptions = {
  result: ScanResult;
  explanation: AIExplanation | null;
  language: Language;
  t: Translations;
};

type PdfPage = {
  canvas: HTMLCanvasElement;
  ctx: CanvasRenderingContext2D;
  y: number;
  currentY: number;
  pageHeight: number;
  margins: number;
  sectionSpacing: number;
  addPage: () => PdfPage;
  ensureSpace: (height: number) => PdfPage;
  measureWrappedText: (
    text: string,
    maxWidth: number,
    size: number,
    weight: number,
  ) => number;
};

const PAGE_WIDTH = 1240;
const PAGE_HEIGHT = 1754;
const MARGIN = 64;
const CONTENT_WIDTH = PAGE_WIDTH - MARGIN * 2;
const CARD_RADIUS = 24;
const COMPACT_GAP = 16;
/** Trailing gap after the last executive recommendation card only (reduces final-page orphan space). */
const EXEC_SECTION_FINAL_GAP = 10;
const SCANNER_ENGINE_VERSION = "v1.5";
const INTELLIGENCE_ENGINE_VERSION = "v1.5";
const PAGE_BOTTOM_SAFE = 64;
const PDF_CODE_SNIPPET_MAX_LINES = 14;
const PDF_CODE_SNIPPET_MAX_CHARS = 1400;
const PDF_CODE_LINE_HEIGHT = 17;
const PDF_CODE_FONT_SIZE = 12;
const PDF_METADATA_TITLE = "CyberGurdian AI Security Report";

type Severity = "critical" | "high" | "medium" | "low" | "informational";

type RemediationItem = {
  severity: Severity;
  title: string;
  why: string;
  impact: string;
  likelihood: string;
  attackSurface: string;
  complexity: string;
  time: string;
  standards: string[];
  remediationEntryId?: string;
};

function sanitize(value: unknown) {
  if (typeof value !== "string") {
    return "";
  }

  return value
    .replace(/<[^>]*>/g, "")
    .replace(/[\u0000-\u001f\u007f]/g, " ")
    .replace(/\s+/g, " ")
    .trim();
}

function escapePdfString(value: string) {
  return value.replace(/\\/g, "\\").replace(/\"/g, "\\").replace(/\\"/g, "\\");
}

function getThreatColor(threatLevel: string) {
  if (threatLevel === "low") {
    return "#34d399";
  }

  if (threatLevel === "medium") {
    return "#fbbf24";
  }

  return "#f87171";
}

function remainingInnerSpace(page: PdfPage): number {
  return page.pageHeight - page.margins - page.y;
}

function ensurePageFitsMinHeight(
  pages: PdfPage[],
  language: Language,
  minBodyHeight: number,
): PdfPage {
  let page = pages[pages.length - 1];

  if (remainingInnerSpace(page) < minBodyHeight + PAGE_BOTTOM_SAFE) {
    page = createPage(language);
    pages.push(page);
  }

  return page;
}

function estimateTwoColumnPairHeight(
  page: PdfPage,
  left: [string, string],
  right: [string, string],
): number {
  const columnGap = COMPACT_GAP;
  const columnWidth = (CONTENT_WIDTH - columnGap) / 2;
```

```

const innerWidth = columnWidth - 40;

const measureCard = ([label, value]: [string, string]) => {
  const labelHeight = page.measureWrappedText(label, innerWidth, 15, 700);
  const valueHeight = page.measureWrappedText(value, innerWidth, 19, 700);

  return Math.max(82, 38 + labelHeight + valueHeight);
};

return Math.max(measureCard(left), measureCard(right)) + COMPACT_GAP;
}

function normalizeCodeSnippetText(code: string): string {
  return code.replace(/\r\n/g, "\n").replace(/\u00a0/g, " ").trimEnd();
}

function truncatePdfCodeSnippet(code: string): string {
  const normalized = normalizeCodeSnippetText(code);
  const lines = normalized.split("\n");
  let take = Math.min(lines.length, PDF_CODE_SNIPPET_MAX_LINES);
  let body = lines.slice(0, take).join("\n");

  while (body.length > PDF_CODE_SNIPPET_MAX_CHARS && take > 1) {
    take -= 1;
    body = lines.slice(0, take).join("\n");
  }

  if (body.length > PDF_CODE_SNIPPET_MAX_CHARS) {
    body = `${body.slice(0, PDF_CODE_SNIPPET_MAX_CHARS - 1)}?`;
  }

  if (lines.length > take) {
    body += "\n?";
  }

  return body;
}

function setMonospaceFont(ctx: CanvasRenderingContext2D, size: number, weight = 500) {
  ctx.font = `${weight} ${size}px ui-monospace, "Cascadia Mono", Consolas, "Courier New", monospace`;
}

function wrapMonospaceLine(ctx: CanvasRenderingContext2D, line: string, maxWidth: number): string[] {
  if (!line.length) {
    return [""];
  }

  if (ctx.measureText(line).width <= maxWidth) {
    return [line];
  }

  const segments: string[] = [];
  let rest = line;

  while (rest.length > 0) {
    let end = rest.length;

    while (end > 1 && ctx.measureText(rest.slice(0, end)).width > maxWidth) {
      end -= 1;
    }

    if (end <= 0) {
      end = 1;
    }

    segments.push(rest.slice(0, end));
    rest = rest.slice(end);
  }

  return segments;
}

function measureMonospaceCodeBlockHeight(
  snippet: string,
  maxWidth: number,
  ctx: CanvasRenderingContext2D,
): number {
  const normalized = normalizeCodeSnippetText(snippet);
  const lines = normalized.split("\n");
  const visible = lines.slice(0, PDF_CODE_SNIPPET_MAX_LINES);
  setMonospaceFont(ctx, PDF_CODE_FONT_SIZE, 500);

  let totalLines = 0;

  for (const line of visible) {
    totalLines += wrapMonospaceLine(ctx, line, maxWidth).length;
  }

  if (lines.length > PDF_CODE_SNIPPET_MAX_LINES) {
    totalLines += 1;
  }

  return Math.max(PDF_CODE_LINE_HEIGHT, totalLines * PDF_CODE_LINE_HEIGHT);
}

function drawMonospaceCodeBlock(
  page: PdfPage,
  snippet: string,
  x: number,
  startY: number,
  maxWidth: number,
): number {
  const { ctx } = page;
  const normalized = normalizeCodeSnippetText(snippet);
  const lines = normalized.split("\n");
  const visible = lines.slice(0, PDF_CODE_SNIPPET_MAX_LINES);
  setMonospaceFont(ctx, PDF_CODE_FONT_SIZE, 500);
  ctx.save();
  ctx.textBaseline = "top";
  ctx.textAlign = "left";
  ctx.direction = "ltr";

  let y = startY;

  for (const line of visible) {
    const segments = wrapMonospaceLine(ctx, line, maxWidth);

    for (const segment of segments) {
      ctx.fillStyle = "#cbd5e1";
    }
  }
}

```

```

        ctx.fillText(segment, x, y);
        y += PDF_CODE_LINE_HEIGHT;
    }
}

if (lines.length > PDF_CODE_SNIPPET_MAX_LINES) {
    ctx.fillStyle = "#94a3b8";
    ctx.fillText("?", x, y);
    y += PDF_CODE_LINE_HEIGHT;
}

ctx.restore();

return y - startY;
}

function mapFindingSeverityToRiskBucket(severity: unknown): "high" | "medium" | "low" {
    const normalized = String(severity ?? "").trim().toLowerCase();

    if (normalized === "critical" || normalized === "high") {
        return "high";
    }

    if (normalized === "medium") {
        return "medium";
    }

    return "low";
}

type SeverityRow = { severity: unknown };

function severityDistributionCounts(rows: readonly SeverityRow[]): {
    high: number;
    medium: number;
    low: number;
} {
    let high = 0;
    let medium = 0;
    let low = 0;

    for (const row of rows) {
        const bucket = mapFindingSeverityToRiskBucket(row.severity);

        if (bucket === "high") {
            high += 1;
        } else if (bucket === "medium") {
            medium += 1;
        } else {
            low += 1;
        }
    }

    const total = rows.length;
    const sum = high + medium + low;

    if (total > 0 && sum !== total) {
        low += total - sum;
    }

    return { high, medium, low };
}

/**
 * Risk donut uses the same severity signals as the rendered PDF:
 * prefer live {@link ScanResult.findings} (Critical Findings). When that array is
 * empty?common on strong sites?executive recommendation cards still render from
 * {@link buildRemediationItems}; those items supply severities so the donut is not all zeros.
 */
function pdfRiskDistributionCounts(
    result: ScanResult,
    language: Language,
    t: Translations,
): {
    high: number;
    medium: number;
    low: number;
} {
    const findingsRows = result.findings ?? [];
    const fromFindings = severityDistributionCounts(findingsRows);

    if (fromFindings.high + fromFindings.medium + fromFindings.low > 0) {
        return fromFindings;
    }

    const remediationItems = buildRemediationItems(result, language, t);

    return severityDistributionCounts(remediationItems);
}

function measureExecutiveRecommendationCardHeight(
    item: RemediationItem,
    index: number,
    remediationEntry: RemediationEntry | undefined,
    page: PdfPage,
    t: Translations,
    language: Language,
    bodyWidth: number,
    layout?: { isFinalCard?: boolean },
): number {
    const titleHeight = page.measureWrappedText(`${index + 1}. ${item.title}`, CONTENT_WIDTH - 210, 22, 900);
    const whyHeight = page.measureWrappedText(`${t.pdfWhyItMatters}: ${item.why}`, bodyWidth, 18, 500);

    const remediationHeight =
        remediationEntry !== undefined
            ? measureRemediationSectionHeight(remediationEntry, language, page, bodyWidth)
            : 0;

    const impactLine = `${t.pdfEstimatedImpact}: ${item.impact} ? ${t.pdfExploitLikelihood}: ${item.likelihood} ? ${t.pdfAttackSurfaceAffected}: ${item.attackSurface}`;
    const standardsLine = `${t.pdfFixComplexity}: ${item.complexity} ? ${t.pdfEstimatedFixTime}: ${item.time} ? ${t.pdfAffectedStandards}: ${item.standards.join(", ")}`;

    const impactMeasured = page.measureWrappedText(impactLine, bodyWidth, 15, 800);
    const standardsMeasured = page.measureWrappedText(standardsLine, bodyWidth, 15, 800);
    const footerBlock = impactMeasured + standardsMeasured + 28;

    const top = 52 + titleHeight + whyHeight + 14;

```

```

const bottomInset = layout?.isFinalCard ? 12 : 22;
return Math.max(220, top + remediationHeight + 6 + footerBlock + bottomInset);
}

function pickExecutiveRemediationCode(entry: RemediationEntry): string | undefined {
const vercel = entry.codeExamples.vercel?.trim();
const nginx = entry.codeExamples.nginx?.trim();

if (vercel) {
return truncatePdfCodeSnippet(vercel);
}

if (nginx) {
return truncatePdfCodeSnippet(nginx);
}

return undefined;
}

function remediationDifficultyLabel(entry: RemediationEntry, t: Translations): string {
if (entry.difficulty === "easy") {
return t.easy;
}

if (entry.difficulty === "medium") {
return t.moderate;
}

return t.advanced;
}

function remediationRiskReductionLabel(
entry: RemediationEntry,
t: Translations,
): string {
if (entry.riskReduction === "high") {
return t.pdfRiskReductionHigh;
}

if (entry.riskReduction === "medium") {
return t.pdfRiskReductionMedium;
}

return t.pdfRiskReductionLow;
}

function measureRemediationSectionHeight(
remediation: RemediationEntry,
language: Language,
page: PdfPage,
bodyWidth: number,
): number {
const summary = remediation.explanation.simple[language];
const summaryHeight = page.measureWrappedText(summary, bodyWidth, 14, 500);
const code = pickExecutiveRemediationCode(remediation);
const codeInnerWidth = bodyWidth - 8;
const codeBlockHeight = code
? 20 + measureMonospaceCodeBlockHeight(code, codeInnerWidth, page.ctx) + 14
: 0;

return 14 + 26 + summaryHeight + 12 + 26 + codeBlockHeight;
}

function getSeverityColor(severity: string) {
if (severity === "critical") {
return "#dc2626";
}

if (severity === "high") {
return "#ef4444";
}

if (severity === "medium") {
return "#f59e0b";
}

return "#22d3ee";
}

function getInfrastructureTrustLabel(result: ScanResult, t: Translations) {
if (result.threatLevel === "critical" || result.threatLevel === "high") {
return t.lowInfrastructureTrust;
}

if (
result.intelligence.reputation === "trusted" &&
result.ssl.valid &&
result.confidence >= 80
) {
return t.highInfrastructureTrust;
}

return t.moderateInfrastructureTrust;
}

function getThreatLabel(threatLevel: string, t: Translations) {
if (threatLevel === "low") {
return t.lowThreat;
}

if (threatLevel === "medium") {
return t.mediumThreat;
}

return t.highThreat;
}

function getThreatDetected(threatLevel: string, t: Translations) {
if (threatLevel === "low") {
return t.lowThreatDetected;
}

if (threatLevel === "medium") {
return t.mediumThreatDetected;
}

return t.highThreatDetected;
}

```

```

function getReputationPdfRows(result: ScanResult, language: Language) {
    const reputation = result.reputation;
    const note =
        language === "ar"
            ? "????? ?????? ?? ?????? ???????????? ?????? ?????? ????????"
            : "Reputation results are intelligence signals, not deterministic proof.";

    if (!reputation) {
        return [
            [
                language === "ar" ? "??? VirusTotal" : "VirusTotal reputation",
                language === "ar" ? "??? ??????" : "Unavailable",
            ],
            [language === "ar" ? "??????? ??????" : "Reputation note", note],
        ] satisfies Array<[string, string]>;
    }

    return [
        [
            language === "ar" ? "??? ??????" : "Reputation verdict",
            reputation.verdict,
        ],
        [
            language === "ar" ? "????????? ??????" : "Malicious detections",
            `${reputation.malicious}/${reputation.totalVendors}`,
        ],
        [
            language === "ar" ? "????????? ??????" : "Suspicious detections",
            `${reputation.suspicious}/${reputation.totalVendors}`,
        ],
        [language === "ar" ? "??????? ??????" : "Reputation note", note],
    ] satisfies Array<[string, string]>;
}

function getExplanation(options: PdfOptions) {
    return options.explanation?.[options.language];
}

function createPage(language: Language): PdfPage {
    const canvas = document.createElement("canvas");
    canvas.width = PAGE_WIDTH;
    canvas.height = PAGE_HEIGHT;

    const ctx = canvas.getContext("2d");

    if (!ctx) {
        throw new Error("canvas_context_unavailable");
    }

    ctx.fillStyle = "#020617";
    ctx.fillRect(0, 0, PAGE_WIDTH, PAGE_HEIGHT);
    ctx.direction = language === "ar" ? "rtl" : "ltr";
    ctx.textBaseline = "top";

    const page = {
        canvas,
        ctx,
        y: MARGIN,
        currentY: MARGIN,
        pageHeight: PAGE_HEIGHT,
        margins: MARGIN,
        sectionSpacing: 20,
        addPage: () => createPage(language),
        ensureSpace: (height: number) => {
            if (page.y + height > PAGE_HEIGHT - MARGIN) {
                return createPage(language);
            }
        },
        return page;
    },
    measureWrappedText: (
        text: string,
        maxWidth: number,
        size: number,
        weight: number,
    ) => {
        setFont(ctx, size, weight, language);
        return wrapText(ctx, text, maxWidth).length * size * (language === "ar" ? 1.55 : 1.32);
    },
};

return page;
}

function roundedRect(
    ctx: CanvasRenderingContext2D,
    x: number,
    y: number,
    width: number,
    height: number,
    radius = CARD_RADIUS,
) {
    ctx.beginPath();
    ctx.moveTo(x + radius, y);
    ctx.lineTo(x + width - radius, y);
    ctx.quadraticCurveTo(x + width, y, x + width, y + radius);
    ctx.lineTo(x + width, y + height - radius);
    ctx.quadraticCurveTo(x + width, y + height, x + width - radius, y + height);
    ctx.lineTo(x + radius, y + height);
    ctx.quadraticCurveTo(x, y + height, x, y + height - radius);
    ctx.lineTo(x, y + radius);
    ctx.quadraticCurveTo(x, y, x + radius, y);
    ctx.closePath();
}

function fillCard(
    ctx: CanvasRenderingContext2D,
    x: number,
    y: number,
    width: number,
    height: number,
    borderColor = "#1e3a5f",
) {
    roundedRect(ctx, x, y, width, height);
    ctx.fillStyle = "#0f172a";
    ctx.fill();
    ctx.strokeStyle = borderColor;
    ctx.lineWidth = 2;
}

```

```

    ctx.stroke();
}

function setFont(
  ctx: CanvasRenderingContext2D,
  size: number,
  weight: number,
  language: Language,
) {
  const family =
    language === "ar"
      ? "Cairo, Inter, Arial, sans-serif"
      : "Inter, Cairo, Arial, sans-serif";
  ctx.font = `${weight} ${size}px ${family}`;
}

function wrapText(
  ctx: CanvasRenderingContext2D,
  text: string,
  maxWidth: number,
) {
  const words = sanitize(text).split(" ");
  const lines: string[] = [];
  let line = "";

  for (const word of words) {
    const testLine = line ? `${line} ${word}` : word;

    if (ctx.measureText(testLine).width > maxWidth && line) {
      lines.push(line);
      line = word;
    } else {
      line = testLine;
    }
  }

  if (line) {
    lines.push(line);
  }

  return lines;
}

function drawText(
  page: PdfPage,
  text: string,
  x: number,
  y: number,
  maxWidth: number,
  options: {
    size?: number;
    weight?: number;
    color?: string;
    lineHeight?: number;
    language: Language;
    align?: CanvasTextAlign;
  },
) {
  const {
    size = 28,
    weight = 400,
    color = "#cbd5e1",
    lineHeight = size * 1.45,
    language,
    align = language === "ar" ? "right" : "left",
  } = options;

  const { ctx } = page;
  const direction = language === "ar" ? "rtl" : "ltr";
  const drawX =
    align === "right"
      ? x + maxWidth
      : align === "center"
      ? x + maxWidth / 2
      : x;

  ctx.save();
  setFont(ctx, size, weight, language);
  ctx.textAlign = align;
  ctx.direction = direction;
  ctx.textBaseline = "top";

  const lines = wrapText(ctx, text, maxWidth);

  lines.forEach((line, index) => {
    // Canvas state is mutable; set text color immediately before every draw.
    ctx.fillStyle = color;
    ctx.textAlign = align;
    ctx.direction = direction;
    ctx.fillText(line, drawX, y + index * lineHeight, maxWidth);
  });

  ctx.direction = "ltr";
  ctx.textAlign = "left";
  ctx.restore();

  return lines.length * lineHeight;
}

function drawSectionTitle(
  page: PdfPage,
  title: string,
  language: Language,
  accent = "#2d3e50",
) {
  const { ctx } = page;
  ctx.fillStyle = accent;
  roundedRect(ctx, MARGIN, page.y, 10, 36, 5);
  ctx.fill();
  drawText(page, title, MARGIN + 24, page.y - 1, CONTENT_WIDTH - 24, {
    size: 27,
    weight: 800,
    color: "#f8f9fc",
    language,
  });
  page.y += 52;
}

function drawSeverityBadge(

```

```

page: PdfPage,
severity: string,
x: number,
y: number,
language: Language,
) {
  const color = getSeverityColor(severity);
  const width = 138;
  const height = 36;

  roundedRect(page.ctx, x, y, width, height, 18);
  page.ctx.fillStyle = `${color}22`;
  page.ctx.fill();
  page.ctx.strokeStyle = color;
  page.ctx.lineWidth = 2;
  page.ctx.stroke();
  drawText(page, severity.toUpperCase(), x + 16, y + 8, width - 32, {
    size: 15,
    weight: 900,
    color,
    language,
    align: "center",
    lineHeight: 18,
  });
}

function ensureSpace(pages: PdfPage[], language: Language, needed: number) {
  let page = pages[pages.length - 1];

  if (page.y + needed > PAGE_HEIGHT - MARGIN) {
    page = createPage(language);
    pages.push(page);
  }

  return page;
}

function drawKeyValueGrid(
  pages: PdfPage[],
  language: Language,
  rows: Array<[string, string]>,
) {
  let page = ensureSpace(pages, language, 180);
  const columnGap = COMPACT_GAP;
  const columnWidth = (CONTENT_WIDTH - columnGap) / 2;
  const rowGap = COMPACT_GAP;
  let rowHeight = 0;
  let rowStartY = page.y;

  rows.forEach(([label, value], index) => {
    const innerWidth = columnWidth - 40;
    const labelHeight = page.measureWrappedText(label, innerWidth, 15, 700);
    const valueHeight = page.measureWrappedText(value, innerWidth, 19, 700);
    const cardHeight = Math.max(82, 38 + labelHeight + valueHeight);
    page = ensureSpace(pages, language, cardHeight + rowGap);
    const column = index % 2;
    if (column === 0) {
      rowStartY = page.y;
      rowHeight = cardHeight;
    } else {
      rowHeight = Math.max(rowHeight, cardHeight);
    }
    const x = MARGIN + column * (columnWidth + columnGap);
    const y = rowStartY;

    fillCard(page.ctx, x, y, columnWidth, cardHeight, "#1e3a5f");
    drawText(page, label, x + 20, y + 16, innerWidth, {
      size: 15,
      weight: 700,
      color: "#67e8f9",
      language,
    });
    drawText(page, value, x + 20, y + 24 + labelHeight, innerWidth, {
      size: 19,
      weight: 700,
      color: "#f8fafc",
      language,
      lineHeight: language === "ar" ? 28 : 24,
    });

    if (column === 1 || index === rows.length - 1) {
      page.y = rowStartY + rowHeight + rowGap;
      page.currentY = page.y;
    }
  });
}

function drawCompactMetadataRows(
  pages: PdfPage[],
  language: Language,
  rows: Array<[string, string]>,
  columns = 4,
) {
  let page = ensureSpace(pages, language, 130);
  const gap = 12;
  const columnWidth = (CONTENT_WIDTH - gap * (columns - 1)) / columns;
  const innerWidth = columnWidth - 28;
  const rowGap = 12;
  let rowHeight = 0;
  let rowStartY = page.y;

  rows.forEach(([label, value], index) => {
    const column = index % columns;
    const labelHeight = page.measureWrappedText(label, innerWidth, 13, 700);
    const valueHeight = page.measureWrappedText(value, innerWidth, 17, 800);
    const cardHeight = Math.max(72, 32 + labelHeight + valueHeight);

    page = ensureSpace(pages, language, cardHeight + rowGap);

    if (column === 0) {
      rowStartY = page.y;
      rowHeight = cardHeight;
    } else {
      rowHeight = Math.max(rowHeight, cardHeight);
    }

    const x = MARGIN + column * (columnWidth + gap);
    const y = rowStartY;
  });
}

```

```

fillCard(page.ctx, x, y, columnWidth, cardHeight, "#1e3a5f");
drawText(page, label, x + 14, y + 12, innerWidth, {
  size: 13,
  weight: 700,
  color: "#67e8f9",
  language,
  lineHeight: 16,
});
drawText(page, value, x + 14, y + 26 + labelHeight, innerWidth, {
  size: 17,
  weight: 800,
  color: "#f8fafc",
  language,
  lineHeight: language === "ar" ? 24 : 21,
});
if (column === columns - 1 || index === rows.length - 1) {
  page.y = rowStartY + rowHeight + rowGap;
  page.currentY = page.y;
}
});
}
}

function drawGauge(
  page: PdfPage,
  x: number,
  y: number,
  value: number,
  label: string,
  color: string,
  language: Language,
) {
  const cardSize = 240;
  const radius = 84;
  const centerX = x + cardSize / 2;
  const centerY = y + 118;
  const start = Math.PI * 0.8;
  const end = Math.PI * 2.2;
  const progress = start + (end - start) * Math.max(0, Math.min(100, value)) / 100;

  fillCard(page.ctx, x, y, cardSize, cardSize, color);
  page.ctx.lineCap = "round";
  page.ctx.lineWidth = 18;
  page.ctx.strokeStyle = "#1e293b";
  page.ctx.beginPath();
  page.ctx.arc(centerX, centerY, radius, start, end);
  page.ctx.stroke();
  page.ctx.strokeStyle = color;
  page.ctx.beginPath();
  page.ctx.arc(centerX, centerY, radius, start, progress);
  page.ctx.stroke();
  page.ctx.lineCap = "butt";
  drawText(page, `${value}%`, x + 30, y + 78, cardSize - 60, {
    size: 38,
    weight: 900,
    color: "#f8fafc",
    language,
    align: "center",
  });
  drawText(page, label, x + 24, y + 160, cardSize - 48, {
    size: 16,
    weight: 700,
    color: "#cbd5e1",
    language,
    align: "center",
    lineHeight: 21,
  });
}

function drawBarChart(
  page: PdfPage,
  x: number,
  y: number,
  width: number,
  title: string,
  rows: Array<{ label: string; value: number; color: string }>,
  language: Language,
) {
  const height = 78 + rows.length * 50;

  fillCard(page.ctx, x, y, width, height, "#1e3a5f");
  drawText(page, title, x + 24, y + 20, width - 48, {
    size: 22,
    weight: 900,
    color: "#f8fafc",
    language,
  });

  rows.forEach((row, index) => {
    const rowY = y + 64 + index * 50;
    drawText(page, row.label, x + 24, rowY, width - 48, {
      size: 15,
      weight: 700,
      color: "#cbd5e1",
      language,
      lineHeight: 22,
    });
    page.ctx.fillStyle = "#1e293b";
    roundedRect(page.ctx, x + 24, rowY + 24, width - 92, 12, 6);
    page.ctx.fill();
    page.ctx.fillStyle = row.color;
    roundedRect(
      page.ctx,
      x + 24,
      rowY + 24,
      Math.max(8, (width - 92) * Math.min(100, row.value) / 100),
      12,
      6,
    );
    page.ctx.fill();
    drawText(page, `${row.value}%`, x + width - 58, rowY + 16, 38, {
      size: 15,
      weight: 800,
      color: row.color,
      language,
      align: "right",
    });
  });
}
}

```



```

function drawDonutChart(
    page: PdfPage,
    x: number,
    y: number,
    title: string,
    segments: Array<{ label: string; value: number; color: string }>,
    language: Language,
) {
    fillCard(page.ctx, x, y, 340, 260, "#1e3a5f");
    drawText(page, title, x + 24, y + 18, 292, {
        size: 21,
        weight: 900,
        color: "#f8fafc",
        language,
    });

    const total = Math.max(1, segments.reduce((sum, segment) => sum + segment.value, 0));
    let angle = -Math.PI / 2;
    const centerX = x + 108;
    const centerY = y + 152;

    segments.forEach((segment) => {
        const slice = (segment.value / total) * Math.PI * 2;
        page.ctx.beginPath();
        page.ctx.moveTo(centerX, centerY);
        page.ctx.fillStyle = segment.color;
        page.ctx.arc(centerX, centerY, 68, angle, angle + slice);
        page.ctx.closePath();
        page.ctx.fill();
        angle += slice;
    });

    page.ctx.fillStyle = "#0f172a";
    page.ctx.beginPath();
    page.ctx.arc(centerX, centerY, 38, 0, Math.PI * 2);
    page.ctx.fill();

    segments.forEach((segment, index) => {
        const legendY = y + 92 + index * 36;
        page.ctx.fillStyle = segment.color;
        roundedRect(page.ctx, x + 204, legendY + 4, 16, 16, 5);
        page.ctx.fill();
        drawText(page, `${segment.label}: ${segment.value}`, x + 228, legendY, 88, {
            size: 14,
            weight: 700,
            color: "#cbd5e1",
            language,
            lineHeight: 20,
        });
    });
}

function drawParagraphCard(
    pages: PdfPage[],
    language: Language,
    title: string,
    body: string,
    accent = "#22d3ee",
) {
    let page = ensureSpace(pages, language, 150);
    const innerWidth = CONTENT_WIDTH - 52;
    const titleHeight = page.measureWrappedText(title, innerWidth, 21, 800);
    const bodyLines = wrapText(page.ctx, body, innerWidth);
    const bodyLineHeight = language === "ar" ? 32 : 27;
    const height = Math.max(128, 58 + titleHeight + bodyLines.length * bodyLineHeight);

    page = ensureSpace(pages, language, height + COMPACT_GAP);
    fillCard(page.ctx, MARGIN, page.y, CONTENT_WIDTH, height, "#1e3a5f");
    drawText(page, title, MARGIN + 26, page.y + 20, innerWidth, {
        size: 21,
        weight: 800,
        color: accent,
        language,
    });
    drawText(page, body, MARGIN + 26, page.y + 36 + titleHeight, innerWidth, {
        size: 19,
        weight: 400,
        color: "#dbeeaf",
        language,
        lineHeight: bodyLineHeight,
    });
    page.y += height + COMPACT_GAP;
    page.currentY = page.y;
}

function buildComplianceMap(issue: string) {
    const normalized = issue.toLowerCase();
    const standards = new Set<string>();

    if (
        normalized.includes("header") ||
        normalized.includes("content-security-policy") ||
        normalized.includes("x-frame-options")
    ) {
        standards.add("OWASP A05:2021");
        standards.add("OWASP ASVS V14");
        standards.add("CIS Control 4");
    }

    if (normalized.includes("tls") || normalized.includes("certificate")) {
        standards.add("OWASP ASVS V9");
        standards.add("NIST SC-13");
        standards.add("ISO 27001 A.8.24");
    }

    if (
        normalized.includes("domain") ||
        normalized.includes("phishing") ||
        normalized.includes("typosquatting")
    ) {
        standards.add("OWASP A06:2021");
        standards.add("CIS Control 9");
    }

    if (normalized.includes("redirect")) {
        standards.add("OWASP A01:2021");
        standards.add("CIS Control 16");
    }
}

```

```

    if (standards.size === 0) {
        standards.add("NIST RA-5");
        standards.add("ISO 27001 A.8.8");
    }

    return Array.from(standards);
}

function recommendationText(
    language: Language,
    en: string,
    ar: string,
) {
    return language === "ar" ? ar : en;
}

function normalizeRecommendationRoot(value: string) {
    const normalized = value
        .toLowerCase()
        .replace(/[\^\\w\s\u0060-\u006FF]/g, " ")
        .replace(/\\s+/g, " ")
        .trim();

    if (
        normalized.includes("infrastructure") &&
        (normalized.includes("fingerprint") ||
            normalized.includes("metadata") ||
            normalized.includes("visible") ||
            normalized.includes("server"))
    ) {
        return "root:infrastructure-exposure";
    }

    if (
        normalized.includes("redirect") ||
        normalized.includes("cross domain")
    ) {
        return "root:redirect";
    }

    return normalized;
}

function isActionableRecommendation(item: RemediationItem) {
    return /^(enable|add|enforce|configure|review|restrict|remove|harden|investigate|verify|reduce)\b/i.test
    (
        item.title.trim(),
    );
}

function dedupeRemediationItems(items: RemediationItem[]) {
    const severityWeight: Record<Severity, number> = {
        critical: 4,
        high: 3,
        medium: 2,
        low: 1,
        informational: 0,
    };
    const uniqueItems = new Map<string, RemediationItem>();

    items.forEach((item) => {
        const root = normalizeRecommendationRoot(`${item.title} ${item.why}`);
        const existing = uniqueItems.get(root);

        if (!existing) {
            uniqueItems.set(root, item);
            return;
        }

        const itemSeverity = severityWeight[item.severity];
        const existingSeverity = severityWeight[existing.severity];
        const itemActionable = isActionableRecommendation(item);
        const existingActionable = isActionableRecommendation(existing);
        const itemDetail = item.title.length + item.why.length;
        const existingDetail = existing.title.length + existing.why.length;

        if (
            itemSeverity > existingSeverity ||
            (itemSeverity === existingSeverity && itemActionable && !existingActionable) ||
            (itemSeverity === existingSeverity &&
                itemActionable === existingActionable &&
                itemDetail > existingDetail)
        ) {
            uniqueItems.set(root, {
                ...item,
                remediationEntryId: item.remediationEntryId ?? existing.remediationEntryId,
            });
        } else {
            uniqueItems.set(root, {
                ...existing,
                remediationEntryId: existing.remediationEntryId ?? item.remediationEntryId,
            });
        }
    });

    return Array.from(uniqueItems.values());
}

function buildRemediationItems(
    result: ScanResult,
    language: Language,
    t: Translations,
): RemediationItem[] {
    const items: RemediationItem[] = [];
    const missingHeaders = Object.entries(result.headers)
        .filter(([_, present]) => !present)
        .map(([header]) => header);

    function add(
        item: Omit<RemediationItem, "standards"> & {
            standards?: string[];
            remediationEntryId?: string;
        },
    ) {
        items.push({
            ...item,
            standards: item.standards ?? buildComplianceMap(item.title),
        });
    }

```

```

}

if (missingHeaders.includes("strict-transport-security")) {
  add({
    severity: "high",
    remediationEntryId: "missing-hsts",
    title: "Enable Strict-Transport-Security immediately",
    why: recommendationText(
      language,
      "Transport-security enforcement reduces downgrade and interception risk on hostile or public netwo
rks.",
      "????? ??? Strict-Transport-Security ??? ????? ?????? ??? ?????? ????????? ?????????? ??? ?????? ?????
? ?? ??????????",
    ),
    impact: "High",
    likelihood: "Medium",
    attackSurface: t.transportSecurity,
    complexity: t.easy,
    time: t.fiveMinutes,
    standards: ["OWASP ASVS V9", "NIST SC-13", "CIS Control 4"],
  });
}

if (missingHeaders.includes("content-security-policy")) {
  add({
    severity: "high",
    remediationEntryId: "missing-csp",
    title: "Add Content-Security-Policy",
    why: recommendationText(
      language,
      "CSP limits script execution paths and reduces the impact of client-side injection conditions.",
      "????? CSP ??? ?????? ?????? ?????? ?????? ?????? ?? ??? ?????? ?????? ?? ??? ??????",
    ),
    impact: "High",
    likelihood: "Medium",
    attackSurface: t.browserSecurity,
    complexity: t.moderate,
    time: t.twoHours,
    standards: ["OWASP A05:2021", "OWASP ASVS V14", "CIS Control 4"],
  });
}

if (missingHeaders.includes("x-frame-options")) {
  add({
    severity: "medium",
    remediationEntryId: "missing-x-frame-options",
    title: "Enforce anti-clickjacking protection",
    why: recommendationText(
      language,
      "X-Frame-Options reduces framing abuse and clickjacking opportunities for sensitive workflows.",
      "????? X-Frame-Options ??? ?????? ?????? ?????? ?????? ?????? Clickjacking ?? ????????? ??????????",
    ),
    impact: "Medium",
    likelihood: "Medium",
    attackSurface: t.browserSecurity,
    complexity: t.easy,
    time: t.fiveMinutes,
    standards: ["OWASP ASVS V14", "CIS Control 4"],
  });
}

if (!result.ssl.valid || result.ssl.weakProtocol || result.ssl.weakCipher) {
  let tlsRemediationId: string | undefined;

  if (result.ssl.selfSigned) {
    tlsRemediationId = "self-signed-ssl";
  } else if (result.ssl.daysLeft <= 0) {
    tlsRemediationId = "expired-ssl";
  } else if (result.ssl.weakProtocol || result.ssl.weakCipher || !result.ssl.valid) {
    tlsRemediationId = "weak-tls";
  }

  add({
    severity: "high",
    remediationEntryId: tlsRemediationId,
    title: "Harden TLS configuration",
    why: recommendationText(
      language,
      "Weak or invalid TLS posture can undermine confidentiality and trust during transport.",
      "?? ?????? ?????? TLS ?????????? ?? ??? ?????????? ??? ?????? ?????????? ?????? ?????? ??????",
    ),
    impact: "High",
    likelihood: "Medium",
    attackSurface: t.transportSecurity,
    complexity: t.moderate,
    time: t.thirtyMinutes,
    standards: ["OWASP ASVS V9", "NIST SC-13", "ISO 27001 A.8.24"],
  });
}

if (result.intelligence.reputation === "suspicious") {
  add({
    severity: "high",
    title: "Investigate Domain trust indicators",
    why: recommendationText(
      language,
      "Suspicious Domain traits can indicate phishing infrastructure, deceptive branding, or low-trust h
osting patterns.",
      "?? ?????? ?????? Domain ?????????? ??? Infrastructure ?????? ?? ?????? ?????? ?? ?????? ?????????? ?????? ???
???",
    ),
    impact: "High",
    likelihood: "Medium",
    attackSurface: t.domainTrust,
    complexity: t.advanced,
    time: t.oneDay,
    standards: ["OWASP A06:2021", "CIS Control 9", "NIST RA-5"],
  });
}

if (result.redirects.suspicious) {
  add({
    severity: "medium",
    remediationEntryId: "suspicious-redirect",
    title: "Review Redirects and cross-Domain flows",
    why: recommendationText(
      language,
      "Unexpected Redirects can route users through lower-trust infrastructure or hide final destination
s.",
    ),
  });
}

```

```

        "?? ???? Redirects ??? ???????? ?????????? ??? Infrastructure ??? ?? ???? ???????? ?????????.",
    ),
    impact: "Medium",
    likelihood: "Medium",
    attackSurface: t.redirectsSurface,
    complexity: t.moderate,
    time: t.thirtyMinutes,
    standards: ["OWASP A01:2021", "CIS Control 16"],
});
}

if (result.technologies.length > 0 || result.meta.server) {
    add({
        severity: "low",
        remediationEntryId: "infrastructure-exposed",
        title: "Reduce Infrastructure fingerprint exposure",
        why: recommendationText(
            language,
            "Visible platform and server metadata helps attackers tune reconnaissance and technology-specific
probing.",
            "????? ?????? ?????? ?Server ??????? ?????????? ??? ?????? ?????????? ?????? ?????? ??????????",
        ),
        impact: "Low",
        likelihood: "Medium",
        attackSurface: t.metadataExposure,
        complexity: t.easy,
        time: t.fiveMinutes,
        standards: ["CIS Control 4", "NIST CM-7"],
    });
}

result.findings.slice(0, 4).forEach((finding) => {
    if (items.some((item) => finding.message.en.toLowerCase().includes(item.title.toLowerCase().slice(0, 1
2)))) {
        return;
    }

    add({
        severity: finding.severity,
        remediationEntryId: finding.id,
        title: finding.message.en,
        why: finding.message[language],
        impact: finding.severity === "high" ? "High" : finding.severity === "medium" ? "Medium" : "Low",
        likelihood: finding.severity === "high" ? "Medium" : "Low",
        attackSurface: t.pdfAttackSurface,
        complexity: finding.severity === "high" ? t.moderate : t.easy,
        time: finding.severity === "high" ? t.thirtyMinutes : t.fiveMinutes,
    });
});

const severityWeight: Record<Severity, number> = {
    critical: 4,
    high: 3,
    medium: 2,
    low: 1,
    informational: 0,
};

return dedupeRemediationItems(items)
    .sort((a, b) => severityWeight[b.severity] - severityWeight[a.severity])
    .slice(0, 8);
}

function drawCover(pages: PdfPage[], options: PdfOptions) {
    const { result, language, t } = options;
    let page = pages[0];
    const threatColor = getThreatColor(result.threatLevel);
    const explanation = getExplanation(options);

    page.ctx.fillStyle = "#020617";
    page.ctx.fillRect(0, 0, PAGE_WIDTH, PAGE_HEIGHT);
    page.ctx.fillStyle = "rgba(34, 211, 238, 0.10)";
    page.ctx.beginPath();
    page.ctx.arc(PAGE_WIDTH - 120, 120, 360, 0, Math.PI * 2);
    page.ctx.fill();

    drawText(page, t.brand, MARGIN, 76, CONTENT_WIDTH, {
        size: 32,
        weight: 900,
        color: "#67e8f9",
        language,
    });
    drawText(page, t.pdfCoverTitle, MARGIN, 126, CONTENT_WIDTH, {
        size: 52,
        weight: 900,
        color: "#f8fafc",
        language,
        lineHeight: 62,
    });
    fillCard(page.ctx, MARGIN, 266, CONTENT_WIDTH, 238, threatColor);
    drawText(page, getThreatDetected(result.threatLevel, t), MARGIN + 32, 298, CONTENT_WIDTH - 64, {
        size: 24,
        weight: 900,
        color: threatColor,
        language,
    });
    drawText(page, getThreatLabel(result.threatLevel, t), MARGIN + 32, 340, CONTENT_WIDTH - 64, {
        size: 54,
        weight: 900,
        color: "ffffff",
        language,
        lineHeight: 60,
    });
    drawText(page, result.meta.finalUrl, MARGIN + 32, 436, CONTENT_WIDTH - 64, {
        size: 19,
        weight: 400,
        color: "#94a3b8",
        language,
        lineHeight: 26,
    });
};

page.y = 542;
drawParagraphCard(pages, language, t.pdfDisclaimerTitle, t.pdfDisclaimer, "#22d3ee");
drawCompactMetadataRows(pages, language, [
    [t.pdfSecurityScore, `${result.score}/100`],
    [t.grade, result.grade],
    [t.confidence, `${result.confidence}%`],
    [t.scanTimestamp, new Date(result.meta.scanTimestamp).toLocaleString(language)],
]);

```

```

    [t.pdfScannerEngine, SCANNER_ENGINE_VERSION],
    [t.pdfIntelligenceEngine, INTELLIGENCE_ENGINE_VERSION],
    [t.pdfThreatClassification, getThreatLabel(result.threatLevel, t)],
    [t.pdfInfrastructureTrust, getInfrastructureTrustLabel(result, t)],
  ]);

  page = ensureSpace(pages, language, 80);
  drawSectionTitle(page, t.pdfExecutiveSummary, language, threatColor);
  drawParagraphCard(
    pages,
    language,
    t.executiveRiskOverview,
    explanation?.executiveRiskOverview ?? t.defaultExecutiveLine,
    threatColor,
  );
}

function drawFindings(pages: PdfPage[], options: PdfOptions) {
  const { result, language, t } = options;
  let page = ensureSpace(pages, language, 100);
  drawSectionTitle(page, t.pdfCriticalFindings, language, "#f87171");

  if (result.findings.length === 0) {
    drawParagraphCard(pages, language, t.pdfCriticalFindings, t.noHighPriorityFindings, "#34d399");
    return;
  }

  result.findings.forEach((finding) => {
    const body = finding.impact?.[language] ?? finding.message[language];
    const standards = buildComplianceMap(finding.message.en);
    const accent = getSeverityColor(finding.severity);
    const bodyHeight = page.measureWrappedText(body, CONTENT_WIDTH - 56, 18, 500);
    const standardsHeight = page.measureWrappedText(
      `${t.pdfAffectedStandards}: ${standards.join(", ")}`,
      CONTENT_WIDTH - 56,
      15,
      700,
    );
    const exploitLine = `${t.pdfExploitLikelihood}: ${finding.severity === "high" ? "Medium" : "Low"} ? ${t.pdfAttackSurfaceAffected}: ${t.pdfAttackSurface} ? ${t.pdfFixComplexity}: ${finding.severity === "high" ? t.moderate : t.easy} ? ${t.pdfEstimatedFixTime}: ${finding.severity === "high" ? t.thirtyMinutes : t.fiveMinutes}`;
    const exploitHeight = page.measureWrappedText(exploitLine, CONTENT_WIDTH - 56, 15, 700);
    const height = Math.max(
      200,
      116 + bodyHeight + exploitHeight + 20 + standardsHeight + 18,
    );

    page = ensureSpace(pages, language, height + COMPACT_GAP);
    const cardY = page.y;

    fillCard(page.ctx, MARGIN, cardY, CONTENT_WIDTH, height, accent);
    drawSeverityBadge(page, finding.severity, MARGIN + 26, cardY + 22, language);
    drawText(page, `${t.pdfTechnicalIssue}`, MARGIN + 184, cardY + 23, CONTENT_WIDTH - 210, {
      size: 16,
      weight: 800,
      color: accent,
      language,
    });
    drawText(page, finding.message.en, MARGIN + 184, cardY + 48, CONTENT_WIDTH - 210, {
      size: 21,
      weight: 900,
      color: "#f8fafc",
      language,
      lineHeight: 26,
    });
    drawText(page, `${t.pdfImpact}: ${body}`, MARGIN + 28, cardY + 100, CONTENT_WIDTH - 56, {
      size: 18,
      weight: 500,
      color: "#d6eaf8",
      language,
      lineHeight: language === "ar" ? 28 : 24,
    });
    drawText(
      page,
      exploitLine,
      MARGIN + 28,
      cardY + 116 + bodyHeight,
      CONTENT_WIDTH - 56,
      {
        size: 15,
        weight: 700,
        color: "#94a3b8",
        language,
        lineHeight: language === "ar" ? 24 : 20,
      },
    );
    drawText(page, `${t.pdfAffectedStandards}: ${standards.join(", ")}`, MARGIN + 28, cardY + height - standardsHeight - 22, CONTENT_WIDTH - 56, {
      size: 15,
      weight: 700,
      color: "#67e8f9",
      language,
      lineHeight: language === "ar" ? 24 : 20,
    });
    page.y = cardY + height + COMPACT_GAP;
  });
}

function drawThreatIntelligence(pages: PdfPage[], options: PdfOptions) {
  const { result, language, t } = options;
  let page = ensureSpace(pages, language, 100);
  drawSectionTitle(page, t.pdfThreatIntelligence, language, "#22d3ee");
  drawKeyValueGrid(pages, language, [
    [t.domain, result.intelligence.domain],
    [t.reputation, result.intelligence.reputation],
    [t.suspiciousTld, result.intelligence.suspiciousTld ? t.yes : t.no],
    [t.phishingKeywords, result.intelligence.phishingKeywords.join(", ") || t.no],
    [t.urlEntropy, String(result.intelligence.entropy)],
    [t.typosquatting, result.intelligence.typosquatting ? t.likely : t.no],
    ...getReputationPdfRows(result, language),
  ]);
}

function drawSecurityAnalytics(pages: PdfPage[], options: PdfOptions) {
  const { result, language, t } = options;
  const confidenceNoteReserve = result.confidence < 70 ? 140 : 0;
  let page = ensureSpace(pages, language, 640 + confidenceNoteReserve);
  const presentHeaders = Object.values(result.headers).filter(Boolean).length;

```

```

const totalHeaders = Object.values(result.headers).length || 1;
const headerCoverage = Math.round((presentHeaders / totalHeaders) * 100);
const { high: highCount, medium: mediumCount, low: lowCount } = pdfRiskDistributionCounts(
  result,
  language,
  t,
);
const browserExposure = Math.round(((totalHeaders - presentHeaders) / totalHeaders) * 100);
const domainExposure = result.intelligence.reputation === "suspicious" ? 85 : result.intelligence.reputation === "neutral" ? 45 : 18;
const infraExposure = Math.min(100, result.technologies.length * 18 + (result.meta.server ? 22 : 0));

drawSectionTitle(page, t.pdfSecurityAnalytics, language, "#22d3ee");
drawGauge(page, MARGIN, page.y, result.score, t.score, getThreatColor(result.threatLevel), language);
drawGauge(page, MARGIN + 264, page.y, result.confidence, t.confidence, "#22d3ee", language);
drawDonutChart(
  page,
  MARGIN + CONTENT_WIDTH - 340,
  page.y,
  t.pdfRiskDistribution,
  [
    { label: "HIGH+", value: highCount, color: "#ef4444" },
    { label: "MEDIUM", value: mediumCount, color: "#f59e0b" },
    { label: "LOW", value: lowCount, color: "#22d3ee" },
  ],
  language,
);
page.y += 284;

if (result.confidence < 70) {
  const noteLineHeight = language === "ar" ? 26 : 22;
  const noteText = t.pdfConfidencePartialVisibility;
  const noteLines = wrapText(page.ctx, noteText, CONTENT_WIDTH - 48).length;
  const noteBlockHeight = noteLines * noteLineHeight + 28;
  page = ensureSpace(pages, language, noteBlockHeight);

  drawText(page, noteText, MARGIN + 24, page.y, CONTENT_WIDTH - 48, {
    size: 15,
    weight: 600,
    color: "#94a3b8",
    language,
    lineHeight: noteLineHeight,
  });
  page.y += noteBlockHeight;
  page.currentY = page.y;
}

drawBarChart(
  page,
  MARGIN,
  page.y,
  CONTENT_WIDTH,
  t.pdfAttackSurface,
  [
    { label: t.pdfHeaderCoverage, value: headerCoverage, color: "#34d399" },
    { label: t.browserSecurity, value: browserExposure, color: "#f59e0b" },
    { label: t.domainTrust, value: domainExposure, color: "#a78bfa" },
    { label: t.pdfInfrastructureExposure, value: infraExposure, color: "#22d3ee" },
  ],
  language,
);
page.y += 292;
}

function drawTlsInfrastructure(pages: PdfPage[], options: PdfOptions) {
  const { result, language, t } = options;
  let page = ensureSpace(pages, language, 100);
  drawSectionTitle(page, t.pdfTlsInfrastructure, language, "#a78bfa");
  drawKeyValueGrid(pages, language, [
    [t.tlsVersion, result.ssl.protocol || t.unknown],
    [t.cipher, result.ssl.cipher || t.unknown],
    [t.issuer, result.ssl.issuer || t.unknown],
    [t.daysLeft, String(result.ssl.daysLeft)],
    [t.infrastructure, result.technologies.join(", ") || t.noFingerprint],
    ["CDN", result.infrastructure.cdn || t.notDisclosed],
    ["WAF", result.infrastructure.waf || t.notDisclosed],
    ["Cloud Provider", result.infrastructure.cloudProvider || t.notDisclosed],
    ["Hosting Provider", result.infrastructure.hostingProvider || t.notDisclosed],
    ["ASN", result.infrastructure.asn || t.notDisclosed],
    [t.server, result.meta.server || t.notDisclosed],
  ]);
}

function drawHeaders(pages: PdfPage[], options: PdfOptions) {
  const { result, language, t } = options;
  let page = ensureSpace(pages, language, 100);
  drawSectionTitle(page, t.pdfSecurityHeaders, language, "#34d399");
  const present = Object.entries(result.headers)
    .filter(([_, value]) => value)
    .map([header] => header)
    .join(", ");
  const missing = Object.entries(result.headers)
    .filter(([_, value]) => !value)
    .map([header] => header)
    .join(", ");

  drawParagraphCard(
    pages,
    language,
    t.present,
    present || t.no,
    "#34d399",
  );
  drawParagraphCard(
    pages,
    language,
    t.missing,
    missing || t.no,
    "#f87171",
  );
  drawParagraphCard(
    pages,
    language,
    t.pdfImpact,
    options.explanation?.[language].attackSurfaceAnalysis ?? t.defaultExecutiveLine,
    "#fbbf24",
  );
}

```

```
function drawMetadataData(pages: PdfPage[], options: PdfOptions) {
  const { result, language, t } = options;
  let page = ensureSpace(pages, language, 100);
  const scanId = `CG-${new Date(result.meta.scanTimestamp).getTime().toString(36).toUpperCase()}-${Math.abs(result.meta.finalUrl.length * 97)}`;

  drawSectionTitle(page, t.pdfScanMetadata, language, "#67e8f9");
  drawCompactMetadataRows(pages, language, [
    [t.pdfScanId, scanId],
    [t.pdfGeneratedBy, `${INTELLIGENCE_ENGINE_VERSION}`],
    [t.pdfScannerEngine, SCANNER_ENGINE_VERSION],
    [t.pdfIntelligenceEngine, INTELLIGENCE_ENGINE_VERSION],
    [t.scanTimestamp, new Date(result.meta.scanTimestamp).toLocaleString(language)],
    [t.confidence, `${result.confidence}%`],
    [t.pdfThreatClassification, getThreatLabel(result.threatLevel, t)],
    [t.pdfInfrastructureTrust, getInfrastructureTrustLabel(result, t)],
    [t.runtime, `${result.meta.responseTime}ms`],
  ]);

  const footerRows: Array<[string, string]> = [
    [t.pdfFinalUrl, result.meta.finalUrl],
    [
      t.pdfRedirectChainSummary,
      result.redirects.chain
        .map((step) => `${step.statusCode} ${step.url}`)
        .join(" | ") || t.no,
    ],
  ];

  page = pages[pages.length - 1];
  const footerPairHeight = estimateTwoColumnPairHeight(page, footerRows[0], footerRows[1]);
  ensurePageFitsMinHeight(pages, language, footerPairHeight + 56);
  drawKeyValueGrid(pages, language, footerRows);
}

function drawExecutiveRecommendations(pages: PdfPage[], options: PdfOptions) {
  const { result, language, t } = options;
  let page = ensureSpace(pages, language, 160);
  const recommendations = buildRemediationItems(result, language, t);
  const bodyWidth = CONTENT_WIDTH - 56;

  drawSectionTitle(page, t.pdfExecutiveRecommendations, language, "#67e8f9");
  drawParagraphCard(
    pages,
    language,
    t.executiveBrief,
    recommendationText(
      language,
      "The following actions are prioritized by severity, exploitability, and expected security impact. They are deterministic recommendations based only on observed scan evidence.",
      "?? ????? ??????? ?????? ?? ?????? ??????? ??????? ?????? ?????? ?????? ?? ?????? ?????? ??",
      "?? ?? ?? ?? ?? ?? ?? ?? ?? ??",
    ),
    "#67e8f9",
  );

  page = pages[pages.length - 1];

  recommendations.forEach((item, index) => {
    const accent = getSeverityColor(item.severity);
    const titleHeight = page.measureWrappedText(`${index + 1}. ${item.title}`, CONTENT_WIDTH - 210, 22, 900);
    const whyHeight = page.measureWrappedText(`${t.pdfWhyItMatters}: ${item.why}`, bodyWidth, 18, 500);
    const isFinalCard = index === recommendations.length - 1;
    const trailingGap = isFinalCard ? EXEC_SECTION_FINAL_GAP : COMPACT_GAP;

    const remediationEntry =
      item.remediationEntryId !== undefined && item.remediationEntryId !== ""
        ? getRemediationById(item.remediationEntryId)
        : undefined;

    const height = measureExecutiveRecommendationCardHeight(
      item,
      index,
      remediationEntry,
      page,
      t,
      language,
      bodyWidth,
      { isFinalCard },
    );

    page = ensureSpace(pages, language, height + trailingGap);
    const cardY = page.y;

    fillCard(page.ctx, MARGIN, cardY, CONTENT_WIDTH, height, accent);
    drawSeverityBadge(page, item.severity, MARGIN + 26, cardY + 22, language);
    drawText(page, `${index + 1}. ${item.title}`, MARGIN + 184, cardY + 22, CONTENT_WIDTH - 210, {
      size: 22,
      weight: 900,
      color: "#f8f4fc",
      language,
      lineHeight: language === "ar" ? 30 : 26,
    });
    const whyY = cardY + 52 + titleHeight;
    drawText(page, `${t.pdfWhyItMatters}: ${item.why}`, MARGIN + 28, whyY, bodyWidth, {
      size: 18,
      weight: 500,
      color: "#d8eafe",
      language,
      lineHeight: language === "ar" ? 28 : 24,
    });

    let cursorY = whyY + whyHeight + 14;

    if (remediationEntry) {
      drawText(page, t.pdfHowToFix, MARGIN + 28, cursorY, bodyWidth, {
        size: 16,
        weight: 800,
        color: "#67e8f9",
        language,
      });
      cursorY += 26;

      const summaryText = remediationEntry.explanation.simple[language];
      const summaryHeight = page.measureWrappedText(summaryText, bodyWidth, 14, 500);

      drawText(page, summaryText, MARGIN + 28, cursorY, bodyWidth, {

```

```

        size: 14,
        weight: 500,
        color: "#d8beaf",
        language,
        lineHeight: language === "ar" ? 24 : 21,
    });
    cursorY += summaryHeight + 12;

    const remediationMeta = `${t.pdfEstimatedFixTime}: ${remediationEntry.estimatedFixTime[language]} ?
    ${t.pdfFixComplexity}: ${remediationDifficultyLabel(remediationEntry, t)} ? ${t.pdfRiskReduction}: ${r
    emediationRiskReductionLabel(remediationEntry, t)}`;

    drawText(page, remediationMeta, MARGIN + 28, cursorY, bodyWidth, {
        size: 14,
        weight: 700,
        color: "#94a3b8",
        language,
        lineHeight: detailLineHeight,
    });
    cursorY += 26;

    const snippet = pickExecutiveRemediationCode(remediationEntry);

    if (snippet) {
        drawText(page, t.pdfSnippetLabel, MARGIN + 28, cursorY, bodyWidth, {
            size: 12,
            weight: 800,
            color: "#67e8f9",
            language,
        });
        cursorY += 20;
        const codeHeight = drawMonospaceCodeBlock(page, snippet, MARGIN + 32, cursorY, bodyWidth - 8);
        cursorY += codeHeight + 14;
    }
}

cursorY += 6;

drawText(
    page,
    `${t.pdfEstimatedImpact}: ${item.impact} ? ${t.pdfExploitLikelihood}: ${item.likelihood} ? ${t.p
    dfAttackSurfaceAffected}: ${item.attackSurface}`,
    MARGIN + 28,
    cursorY,
    bodyWidth,
    {
        size: 15,
        weight: 800,
        color: "#cbd5e1",
        language,
        lineHeight: detailLineHeight,
    },
);
drawText(
    page,
    `${t.pdfFixComplexity}: ${item.complexity} ? ${t.pdfEstimatedFixTime}: ${item.time} ? ${t.pdfAff
    ectedStandards}: ${item.standards.join(", ")}`,
    MARGIN + 28,
    cursorY + detailLineHeight + 22,
    bodyWidth,
    {
        size: 15,
        weight: 800,
        color: "#67e8f9",
        language,
        lineHeight: detailLineHeight,
    },
);
page.y = cardY + height + trailingGap;
});
}

async function renderPages(options: PdfOptions) {
    if ("fonts" in document) {
        await document.fonts.ready;
    }

    const pages =
        options.language === "ar" ? renderArabicPDF(options) : renderEnglishPDF(options);

    const canvases = pages.map((page) => page.canvas);

    validateCanvases(canvases);

    return canvases;
}

function renderEnglishPDF(options: PdfOptions) {
    const pages = [createPage("en")];

    drawCover(pages, options);
    drawSecurityAnalytics(pages, options);
    drawFindings(pages, options);
    drawThreatIntelligence(pages, options);
    drawTlsInfrastructure(pages, options);
    drawHeaders(pages, options);
    drawMetadata(pages, options);
    drawExecutiveRecommendations(pages, options);

    return pages;
}

function renderArabicPDF(options: PdfOptions) {
    const pages = [createPage("ar")];

    drawCover(pages, options);
    drawSecurityAnalytics(pages, options);
    drawFindings(pages, options);
    drawThreatIntelligence(pages, options);
    drawTlsInfrastructure(pages, options);
    drawHeaders(pages, options);
    drawMetadata(pages, options);
    drawExecutiveRecommendations(pages, options);

    return pages;
}

function validateCanvases(canvases: HTMLCanvasElement[]) {
    if (canvases.length === 0) {

```



```

    throw new Error("pdf_no_pages");
  }

  for (let index = 0; index < canvases.length; index += 1) {
    const canvas = canvases[index];

    if (
      canvas.width !== PAGE_WIDTH ||
      canvas.height !== PAGE_HEIGHT ||
      canvas.width <= 0 ||
      canvas.height <= 0
    ) {
      throw new Error(`pdf_invalid_page_dimensions_${index}`);
    }
  }
}

function base64ToBytes(dataUrl: string) {
  const base64 = dataUrl.split(",")[1] ?? "";
  const binary = atob(base64);
  const bytes = new Uint8Array(binary.length);

  for (let index = 0; index < binary.length; index += 1) {
    bytes[index] = binary.charCodeAtAt(index);
  }

  return bytes;
}

async function canvasToJpegBytes(canvas: HTMLCanvasElement) {
  await new Promise<void>((resolve) => {
    window.requestAnimationFrame(() => resolve());
  });

  const blob = await new Promise<Blob | null>((resolve) => {
    canvas.toBlob(resolve, "image/jpeg", 0.92);
  });

  if (blob && blob.size > 0) {
    return new Uint8Array(await blob.arrayBuffer());
  }

  const fallbackBytes = base64ToBytes(canvas.toDataURL("image/jpeg", 0.92));

  if (fallbackBytes.length === 0) {
    throw new Error("pdf_empty_canvas_image");
  }

  return fallbackBytes;
}

const textEncoder = new TextEncoder();

function encodeAscii(value: string) {
  return textEncoder.encode(value);
}

function byteLength(parts: Uint8Array[]) {
  return parts.reduce((total, part) => total + part.byteLength, 0);
}

function toArrayBuffer(part: Uint8Array) {
  const copy = new Uint8Array(part.byteLength);
  copy.set(part);

  return copy.buffer;
}

function sanitizePdfInfo(value: string) {
  return sanitize(value).replace(/[\^x20-\x7e]/g, "?");
}

type PdfObject = {
  id: number;
  parts: Uint8Array[];
};

async function buildPdfFromCanvases(canvases: HTMLCanvasElement[], title: string) {
  validateCanvases(canvases);

  const objects: PdfObject[] = [];
  const pageObjectIds: number[] = [];

  function addObject(parts: Uint8Array[]) {
    const id = objects.length + 1;
    objects.push({ id, parts });
    return id;
  }

  const catalogId = addObject([encodeAscii("<< /Type /Catalog /Pages 2 0 R >>")]);
  const pagesPlaceholderId = addObject([encodeAscii("<<>>")]);

  for (let index = 0; index < canvases.length; index += 1) {
    const canvas = canvases[index];
    const imageBytes = await canvasToJpegBytes(canvas);

    if (imageBytes.byteLength === 0) {
      throw new Error(`pdf_empty_image_stream_${index}`);
    }

    const imageObjectId = objects.length + 1;
    const contentObjectId = imageObjectId + 1;
    const pageObjectId = imageObjectId + 2;
    const content = encodeAscii(
      `q\n${PAGE_WIDTH} 0 0 ${PAGE_HEIGHT} 0 0 cm\n/Im${imageObjectId} Do\nQ\n`,
    );

    addObject([
      encodeAscii(
        "<< /Type /XObject /Subtype /Image /Width ${PAGE_WIDTH} /Height ${PAGE_HEIGHT} /ColorSpace /Device\nRGB /BitsPerComponent 8 /Filter /DCTDecode /Length ${imageBytes.byteLength} >>\nstream\n",
      ),
      imageBytes,
      encodeAscii("\nendstream"),
    ]);
    addObject([
      encodeAscii("<< /Length ${content.byteLength} >>\nstream\n",
      content,
      encodeAscii("endstream"),
    ]);
  }
}

```

```

    });
    addObject([
      encodeAscii(
        `<< /Type /Page /Parent ${pagesPlaceholderId} 0 R /MediaBox [0 0 ${PAGE_WIDTH} ${PAGE_HEIGHT}] /Re
sources << /XObject << /Im${imageObjectId} ${imageObjectId} 0 R >> >> /Contents ${contentObjectId} 0 R >>`
      ),
    ]);
    pageObjectIds.push(pageObjectId);
  }

  objects[pagesPlaceholderId - 1] = {
    id: pagesPlaceholderId,
    parts: [
      encodeAscii(
        `<< /Type /Pages /Kids [${pageObjectIds
          .map(id => `${id} 0 R`)
          .join(" ")}] /Count ${pageObjectIds.length} >>`,
      ),
    ],
  };

  const infoObjectId = addObject([
    encodeAscii(
      `<< /Title (${escapePdfString(sanitizePdfInfo(PDF_METADATA_TITLE))) /Producer (CyberGurdian AI) >>`
    ),
  ]);

  const outputParts: Uint8Array[] = [
    encodeAscii(`PDF-1.4\n%\xE2\xE3\xCF\xD3\n`),
  ];
  const offsets = [0];

  for (const object of objects) {
    offsets[object.id] = byteLength(outputParts);
    outputParts.push(encodeAscii(`${object.id} 0 obj\n`, ...object.parts, encodeAscii(`\nendobj\n`)));
  }

  const xrefOffset = byteLength(outputParts);
  const xrefRows = [
    `xref\n0 ${objects.length + 1}\n`,
    `0000000000 65535 f \n`,
    ...objects.map((object) => {
      const offset = offsets[object.id];
      return `${String(offset).padStart(10, "0")} 00000 n \n`;
    }),
  ];
  outputParts.push(
    encodeAscii(
      `${xrefRows.join("")}trailer\n<< /Size ${objects.length + 1} /Root ${catalogId} 0 R /Info ${infoObje
ctId} 0 R >>\nstartxref\n${xrefOffset}\n%%EOF\n`,
    ),
  );

  const blob = new Blob(outputParts.map(toArrayBuffer), { type: "application/pdf" });
  await validatePdfBlob(blob, canvases.length);

  return blob;
}

async function validatePdfBlob(blob: Blob, expectedPages: number) {
  if (blob.size < 1024) {
    throw new Error("pdf_blob_too_small");
  }

  const buffer = await blob.arrayBuffer();
  const decoder = new TextDecoder("utf-8", { fatal: false });
  const header = decoder.decode(buffer.slice(0, 8));
  const tail = decoder.decode(buffer.slice(Math.max(0, buffer.byteLength - 32)));

  if (!header.startsWith("%PDF-")) {
    throw new Error("pdf_invalid_header");
  }

  if (!tail.includes("%%EOF")) {
    throw new Error("pdf_missing_eof");
  }

  if (expectedPages <= 0) {
    throw new Error("pdf_expected_pages_invalid");
  }

  const pdfText = decoder.decode(buffer);
  const pageCount = pdfText.match(/\/Type \\/Page\b/g)?.length ?? 0;

  if (pageCount !== expectedPages) {
    throw new Error(`pdf_page_count_mismatch_${pageCount}_${expectedPages}`);
  }
}

function sanitizeFilenamePart(value: string) {
  return value
    .normalize("NFKD")
    .replace(/[\^\x00-\x7F]/g, "")
    .toLowerCase()
    .replace(/[\^a-z0-9]+/g, "-")
    .replace(/^-+|-+$/g, "")
    .slice(0, 80);
}

function getSafeReportFilename(finalUrl: string, scanTimestamp: string) {
  try {
    const hostname = new URL(finalUrl).hostname;
    const domain = sanitizeFilenamePart(hostname.replace(/\./g, "-"));
    const date = new Date(scanTimestamp).toISOString().slice(0, 10);

    if (!domain || !date) {
      return "cybergurdian-ai-security-report.pdf";
    }

    return `cybergurdian-ai-security-report-${domain}-${date}.pdf`;
  } catch {
    return "cybergurdian-ai-security-report.pdf";
  }
}

```

```
export async function generateCyberGuardianPdf(options: PdfOptions) {
  try {
    const canvases = await renderPages(options);
    const blob = await buildPdfFromCanvases(canvases, options.t.pdfCoverTitle);
    const url = URL.createObjectURL(blob);
    const anchor = document.createElement("a");
    const filename = getSafeReportFilename(
      options.result.meta.finalUrl,
      options.result.meta.scanTimestamp,
    );

    anchor.href = url;
    anchor.download = filename;
    document.body.appendChild(anchor);
    anchor.click();
    anchor.remove();

    window.setTimeout(() => URL.revokeObjectURL(url), 30_000);
  } catch (error) {
    throw error;
  }
}
```

lib/scanner.ts

```
import "server-only";

import { intelligenceMap, type IntelligenceFindingId } from "@lib/intelligenceMap";
import { checkDomainReputation } from "@lib/reputation";
import type {
  Finding,
  InfrastructureDetection,
  RedirectStep,
  ScanStageName,
  ScanStageState,
  ScanResult,
  Severity,
  ThreatLevel,
} from "@lib/types";

type ValidationResult = {
  valid: boolean;
  url: string;
  blockedReason?: string;
};

type ValidateUrl = (input: string) => Promise<ValidationResult>;

type FetchStep = {
  response: Response;
  url: string;
  statusCode: number;
};

type CertificateIdentity = {
  CN?: string;
  O?: string;
  OU?: string;
};

type PeerCertificate = {
  issuer?: CertificateIdentity;
  subject?: CertificateIdentity;
  issuerCertificate?: {
    fingerprint256?: string;
  };
  fingerprint256?: string;
  valid_to?: string;
};

type TlsSocketLike = {
  authorized: boolean;
  authorizationError: string | Error | null;
  getPeerCertificate: (detailed: true) => unknown;
  getProtocol: () => string | null;
  getCipher: () => {
    name?: string;
    standardName?: string;
    version?: string;
  };
  once: (event: string, listener: () => void) => unknown;
  removeAllListeners: () => unknown;
  destroy: () => unknown;
};

type TlsModule = {
  connect: (options: {
    host: string;
    port: number;
    servername: string;
    rejectUnauthorized: boolean;
  }) => TlsSocketLike;
};

type PassiveSignals = {
  headerNames: string;
  headerValues: string;
  server: string;
  poweredBy: string;
  via: string;
  cookies: string;
  finalHostname: string;
  redirectHostnames: string[];
};

type Candidate = {
  category: InfrastructureDetection["category"];
  name: string;
  score: number;
  signals: string[];
};

class ScanStageTimeoutError extends Error {
  constructor(public readonly stage: ScanStageName) {
    super(`${stage}_timeout`);
    this.name = "ScanStageTimeoutError";
  }
}

const MAX_REDIRECTS = 5;
const MAX_RESPONSE_BYTES = 1_000_000;
const TRANSIENT_RETRY_ATTEMPTS = 2;
const POSITIVE_BONUS_CAP = 20;
const STAGE_TIMEOUTS: Record<Exclude<ScanStageName, "aiSummary">, number> = {
  dns: 4_000,
  tls: 4_000,
  headers: 6_000,
  redirects: 12_000,
  infrastructure: 1_000,
};

const SCAN_STAGE_NAMES: ScanStageName[] = [
  "dns",
  "tls",
  "headers",
  "redirects",
  "infrastructure",
  "aiSummary",
];

const SECURITY_HEADERS = [
  "strict-transport-security",
  "content-security-policy",
];
```

```

"x-frame-options",
"x-content-type-options",
"referrer-policy",
"permissions-policy",
] as const;

const SCORE_WEIGHTS = {
  tlsInvalid: 22,
  tlsSelfSigned: 20,
  tlsExpired: 24,
  tlsExpiringSoon: 5,
  tlsWeakProtocol: 16,
  tlsWeakCipher: 8,
  missingHeader: 2,
  browserHardeningCluster: 8,
  redirectSuspicious: 7,
  criticalFinding: 28,
  highFinding: 14,
  mediumFinding: 7,
  lowFinding: 2,
  informationalFinding: 0,
  suspiciousReputation: 8,
  suspiciousTld: 4,
  punycode: 6,
  typosquatting: 8,
  activePhishing: 38,
  phishingKeyword: 5,
  reputationMalicious: 20,
  reputationSuspicious: 8,
  excessiveSubdomains: 5,
  highEntropy: 5,
  serverExposure: 6,
  serverError: 5,
} as const;

const TRUSTED_ENTERPRISE_DOMAINS = new Set([
  "amazon.com",
  "cloudflare.com",
  "github.com",
  "google.com",
  "microsoft.com",
  "openai.com",
]);

const TRUSTED_CERTIFICATE_ISSUERS = [
  "let's encrypt",
  "lets encrypt",
  "google trust services",
  "digicert",
  "sectigo",
  "cloudflare",
];

const PROTECTED_EDGE_PROVIDERS = [
  "cloudflare",
  "akamai",
  "fastly",
  "aws cloudfront",
];

const TRUSTED_INFRASTRUCTURE_SIGNALS = [
  "google as15169",
  "cloudflare as13335",
  "amazon as16509",
  "microsoft as8075",
  "fastly",
  "akamai",
];

const SUSPICIOUS_TLDS = new Set([
  "buzz",
  "click",
  "country",
  "cq",
  "gq",
  "info",
  "loan",
  "mom",
  "party",
  "pw",
  "rest",
  "review",
  "ru",
  "stream",
  "support",
  "tk",
  "top",
  "work",
  "xyz",
  "zip",
]);

const PHISHING_KEYWORDS = [
  "account",
  "bank",
  "billing",
  "confirm",
  "crypto",
  "login",
  "password",
  "paypal",
  "secure",
  "signin",
  "support",
  "update",
  "verify",
  "wallet",
];

const TRUSTED_DOMAINS = new Set([
  "cloudflare.com",
  "github.com",
  "google.com",
  "microsoft.com",
  "openai.com",
]);

const BRAND_KEYWORDS = [
  "apple",
  "amazon",

```

```

"cloudflare",
"facebook",
"github",
"google",
"instagram",
"microsoft",
"netflix",
"openai",
"paypal",
"twitter",
"whatsapp",
];

const TYPO_REPLACEMENTS: Record<string, string> = {
  "0": "o",
  "1": "l",
  "3": "e",
  "4": "a",
  "5": "s",
  "@": "a",
};

const FINDING_SCORE_DEDUCTION: Record<Severity, number> = {
  critical: 30,
  high: 15,
  medium: 8,
  low: 3,
  informational: SCORE_WEIGHTS.informationalFinding,
};

function assertNodeRuntime() {
  const runtime = globalThis as typeof globalThis & {
    EdgeRuntime?: string;
    process?: {
      versions?: {
        node?: string;
      };
    };
  };
};

if (
  !runtime.process?.versions?.node ||
  runtime.EdgeRuntime
) {
  throw new Error("node_runtime_required");
}

async function loadTls(): Promise<TlsModule> {
  assertNodeRuntime();

  // Keep Node's TLS module out of client and Edge bundles. The import is
  // intentionally delayed until after the runtime guard has passed.
  const importNodeModule = new Function("specifier", "return import(specifier)") as (
    specifier: string,
  ) => Promise<TlsModule>;

  return importNodeModule("tls");
}

async function loadValidator(): Promise<ValidateUrl> {
  assertNodeRuntime();

  const { validateUrl } = await import("@/lib/validator");

  return validateUrl;
}

function createInitialStages(): Record<ScanStageName, ScanStageState> {
  return Object.fromEntries(
    SCAN_STAGE_NAMES.map((name) => [
      name,
      {
        name,
        status: name === "aiSummary" ? "pending" : "pending",
        durationMs: 0,
      },
    ]),
  ) as Record<ScanStageName, ScanStageState>;
}

function updateStage(
  result: ScanResult,
  name: ScanStageName,
  status: ScanStageState["status"],
  startedAt: number,
  reason?: string,
) {
  result.meta.stages[name] = {
    name,
    status,
    durationMs: Date.now() - startedAt,
    ...(reason ? { reason } : {}),
  };
}

function getErrorReason(error: unknown, fallback: string) {
  if (error instanceof ScanStageTimeoutError) {
    return error.message;
  }

  if (error instanceof DOMException && error.name === "AbortError") {
    return fallback;
  }

  if (error instanceof Error && error.message) {
    return error.message.replace(/^[^a-z0-9_-]/gi, "_").toLowerCase().slice(0, 80);
  }

  return fallback;
}

function isTransientError(error: unknown) {
  if (error instanceof ScanStageTimeoutError) {
    return true;
  }

  if (error instanceof DOMException && error.name === "AbortError") {
    return true;
  }
}

```

```

    }

    if (!(error instanceof Error)) {
        return false;
    }

    return /timeout|abort|econnreset|timedout|eai_again|network|fetch failed/i.test(
        error.message,
    );
}

function combineSignals(primary: AbortSignal, secondary: AbortSignal) {
    const controller = new AbortController();
    const abort = () => controller.abort();

    if (primary.aborted || secondary.aborted) {
        controller.abort();
        return {
            signal: controller.signal,
            cleanup: () => undefined,
        };
    }

    primary.addEventListener("abort", abort, { once: true });
    secondary.addEventListener("abort", abort, { once: true });

    return {
        signal: controller.signal,
        cleanup: () => {
            primary.removeEventListener("abort", abort);
            secondary.removeEventListener("abort", abort);
        },
    };
}

async function withStageTimeout<T>(
    stage: ScanStageName,
    timeoutMs: number,
    task: (signal: AbortSignal) => Promise<T>,
) {
    const controller = new AbortController();
    let timer: ReturnType<typeof setTimeout> | undefined;

    const timeoutPromise = new Promise<never>((_, reject) => {
        timer = setTimeout(() => {
            controller.abort();
            reject(new ScanStageTimeoutError(stage));
        }, timeoutMs);
    });

    try {
        return await Promise.race([task(controller.signal), timeoutPromise]);
    } finally {
        if (timer) {
            clearTimeout(timer);
        }
    }
}

async function retryTransient<T>(
    task: () => Promise<T>,
    attempts = TRANSIENT_RETRY_ATTEMPTS,
) {
    let lastError: unknown;

    for (let attempt = 1; attempt <= attempts; attempt += 1) {
        try {
            return await task();
        } catch (error) {
            lastError = error;

            if (!isTransientError(error) || attempt === attempts) {
                break;
            }
        }
    }

    throw lastError;
}

function createEmptyResult(finalUrl = ""): ScanResult {
    return {
        score: 100,
        grade: "A",
        confidence: 0,
        threatLevel: "low",
        deterministicHash: "",
        ssl: {
            valid: false,
            selfSigned: false,
            issuer: "",
            daysLeft: 0,
            protocol: "",
            cipher: "",
            weakProtocol: false,
            weakCipher: false,
        },
        headers: Object.fromEntries(
            SECURITY_HEADERS.map((header) => [header, false]),
        ),
        intelligence: {
            domain: "",
            reputation: "neutral",
            suspiciousTld: false,
            punycode: false,
            typosquatting: false,
            phishingKeywords: [],
            excessiveSubdomains: false,
            entropy: 0,
            dnsRisk: "low",
            activePhishingIndicators: false,
        },
        infrastructure: {
            cdn: "",
            waf: "",
            hostingProvider: "",
            cloudProvider: "",
            asn: "",
            reverseProxy: "",
        },
    };
}

```

```

        framework: "",
        ipOwner: "",
        confidence: 0,
        detections: [],
        serverExposureScore: 0,
        redirectTrust: "neutral",
    },
    technologies: [],
    reputation: null,
    redirects: {
        chain: [],
        suspicious: false,
    },
    meta: {
        responseTime: 0,
        statusCode: 0,
        finalUrl: "",
        server: "",
        scanTimestamp: new Date().toISOString(),
        stages: createInitialStages(),
    },
    findings: [],
};

function addFinding(
    findings: Finding[],
    severity: Severity,
    en: string,
    ar: string,
    id?: IntelligenceFindingId,
) {
    const intelligence = id ? intelligenceMap[id] : undefined;

    findings.push({
        ...(id ? { id } : {}),
        severity: intelligence?.severity ?? severity,
        message: { en, ar },
        ...(intelligence
            ? {
                impact: intelligence.impact,
                remediation: intelligence.remediation,
            }
            : {}),
    });
}

function calculateGrade(score: number) {
    if (score >= 85) {
        return "A";
    }

    if (score >= 70) {
        return "B";
    }

    if (score >= 55) {
        return "C";
    }

    if (score >= 40) {
        return "D";
    }

    return "F";
}

function calculateThreatLevel(score: number): ThreatLevel {
    if (score < 40) {
        return "critical";
    }

    if (score < 55) {
        return "high";
    }

    if (score < 70) {
        return "medium";
    }

    return "low";
}

function clampScore(score: number) {
    return Math.max(0, Math.min(100, Math.round(score)));
}

function normalizeSignal(value: string) {
    return value.toLowerCase();
}

function isTrustedCertificateIssuer(issuer: string) {
    const normalized = normalizeSignal(issuer);

    return TRUSTED_CERTIFICATE_ISSUERS.some((trustedIssuer) =>
        normalized.includes(trustedIssuer),
    );
}

function isTls13OrNewer(protocol: string) {
    const match = protocol.match(/TLSv?(\d+)\.(\d+)/i);

    if (!match) {
        return false;
    }

    const major = Number(match[1]);
    const minor = Number(match[2]);

    return major > 1 || (major === 1 && minor >= 3);
}

function isTls12OrNewer(protocol: string) {
    const match = protocol.match(/TLSv?(\d+)\.(\d+)/i);

    if (!match) {
        return false;
    }

```



```

const major = Number(match[1]);
const minor = Number(match[2]);

return major > 1 || (major === 1 && minor >= 2);
}

function hasProtectedEdgeProvider(result: ScanResult) {
  const combinedSignals = [
    result.infrastructure.cdn,
    result.infrastructure.waf,
    result.infrastructure.reverseProxy,
    ...result.technologies,
  ]
    .join(" ")
    .toLowerCase();

  return PROTECTED_EDGE_PROVIDERS.some((provider) =>
    combinedSignals.includes(provider),
  );
}

function hasTrustedInfrastructureSignal(result: ScanResult) {
  const combinedSignals = [
    result.infrastructure.asn,
    result.infrastructure.ipOwner,
    result.infrastructure.cdn,
    result.infrastructure.cloudProvider,
    result.infrastructure.hostingProvider,
    result.infrastructure.reverseProxy,
    ...result.infrastructure.detections.map((detection) => detection.name),
    ...result.infrastructure.detections.flatMap((detection) => detection.signals),
    ...result.technologies,
  ]
    .join(" ")
    .toLowerCase();

  return TRUSTED_INFRASTRUCTURE_SIGNALS.some((signal) =>
    combinedSignals.includes(signal),
  );
}

function isScannerVisibilityLimited(result: ScanResult) {
  return Object.values(result.meta.stages).some((stage) => {
    const reason = stage.reason ?? "";

    return (
      reason.includes("unable_to_verify") ||
      reason.includes("anti_bot") ||
      reason.includes("challenge") ||
      reason.includes("headers_timeout") ||
      reason.includes("final_response_unavailable") ||
      stage.status === "timeout" ||
      stage.status === "partial"
    );
  });
}

function isTrustedEnterpriseTarget(result: ScanResult) {
  return TRUSTED_ENTERPRISE_DOMAINS.has(result.intelligence.domain);
}

function hasCriticalTrustOverride(result: ScanResult) {
  return (
    result.reputation?.verdict === "malicious" ||
    result.intelligence.activePhishingIndicators ||
    result.redirects.suspicious ||
    result.ssl.selfSigned ||
    (result.ssl.daysLeft <= 0 && result.meta.finalUrl.startsWith("https://")) ||
    result.findings.some((finding) => finding.severity === "critical")
  );
}

function isKnownMaliciousOrPhishing(result: ScanResult) {
  return (
    result.reputation?.verdict === "malicious" ||
    result.intelligence.activePhishingIndicators
  );
}

function calculatePositiveTrustBonus(result: ScanResult) {
  const bonuses = [
    result.reputation?.verdict === "clean" ? 5 : 0,
    isTls13OrNewer(result.ssl.protocol) ? 5 : 0,
    isTrustedCertificateIssuer(result.ssl.issuer) ? 5 : 0,
    result.intelligence.entropy > 0 && result.intelligence.entropy < 3 ? 3 : 0,
    !result.redirects.suspicious ? 5 : 0,
    hasProtectedEdgeProvider(result) ? 3 : 0,
    hasTrustedInfrastructureSignal(result) ? 5 : 0,
  ];

  return Math.min(
    POSITIVE_BONUS_CAP,
    bonuses.reduce((total, bonus) => total + bonus, 0),
  );
}

function applyScoreSanityProtection(score: number, result: ScanResult) {
  let calibratedScore = score;

  if (isTrustedEnterpriseTarget(result) && !hasCriticalTrustOverride(result)) {
    calibratedScore = Math.max(70, calibratedScore);
  }

  if (isKnownMaliciousOrPhishing(result)) {
    calibratedScore = Math.min(40, calibratedScore);
  }

  return clampScore(calibratedScore);
}

function stableStringify(value: unknown): string {
  if (Array.isArray(value)) {
    return `[${value.map(stableStringify).join(",")}];`
  }

  if (value && typeof value === "object") {
    return `{${Object.entries(value)
      .sort(([left], [right]) => left.localeCompare(right))
      .map(([key, entry]) => `${JSON.stringify(key)}:${stableStringify(entry)}`)
    }}`
  }
}

```

```

    .join(",")}`;
}
return JSON.stringify(value);
}

export function generateDeterministicHash(scanResult: ScanResult) {
    const rawScoringData = {
        ssl: scanResult.ssl,
        headers: scanResult.headers,
        intelligence: scanResult.intelligence,
        infrastructure: scanResult.infrastructure,
        reputation: scanResult.reputation,
        redirects: scanResult.redirects,
        statusCode: scanResult.meta.statusCode,
        stageStatuses: Object.fromEntries(
            Object.entries(scanResult.meta.stages).map(([stage, state]) => [
                stage,
                state.status,
            ]),
        ),
        technologies: scanResult.technologies,
        findingSeverities: scanResult.findings.map((finding) => finding.severity),
    };
    const source = stableStringify(rawScoringData);
    let hash = 0x811c9dc5;

    for (let index = 0; index < source.length; index += 1) {
        hash ^= source.charCodeAt(index);
        hash = Math.imul(hash, 0x01000193);
    }

    return `cg-${(hash >>> 0).toString(16).padStart(8, "0")}`;
}

function calculateConfidence(result: ScanResult) {
    let confidence = 70;
    const stages = Object.values(result.meta.stages);
    const hasTlsData = Boolean(result.ssl.issuer || result.ssl.protocol || result.ssl.cipher);
    const hasHeaderData =
        result.meta.stages.headers.status === "completed" &&
        Object.values(result.headers).length >= SECURITY_HEADERS.length;
    const hasInfrastructureFingerprint =
        result.technologies.length > 0 ||
        result.infrastructure.detections.length > 0 ||
        Boolean(
            result.infrastructure.cdn ||
            result.infrastructure.waf ||
            result.infrastructure.hostingProvider ||
            result.infrastructure.cloudProvider ||
            result.infrastructure.framework ||
            result.infrastructure.reverseProxy,
        );
    const antiBotBlocked =
        result.meta.statusCode === 403 ||
        result.meta.stages.headers.reason?.includes("unable_to_verify") ||
        result.meta.stages.headers.reason?.includes("anti_bot") ||
        result.meta.stages.headers.reason?.includes("challenge");
    const hasTimeoutOrPartial = stages.some(
        (stage) => stage.status === "timeout" || stage.status === "partial",
    );

    if (hasTlsData) {
        confidence += 10;
    }

    if (hasHeaderData) {
        confidence += 8;
    }

    if (result.reputation) {
        confidence += 7;
    }

    if (hasInfrastructureFingerprint) {
        confidence += 5;
    }

    if (antiBotBlocked) {
        confidence -= 15;
    }

    if (hasTimeoutOrPartial) {
        confidence -= 20;
    }

    return Math.max(0, Math.min(97, Math.round(confidence)));
}

function normalizeFindingTitle(value: string) {
    const normalized = value
        .toLowerCase()
        .replace(/[^\w\s\u0600-\u06FF]/g, " ")
        .replace(/\s+/g, " ")
        .trim();

    return normalized
        .replace(/bmissing security header\b/g, " ")
        .replace(/btechnical issue\b/g, " ")
        .replace(/bpreview\b/g, " ")
        .replace(/bdetected\b/g, " ")
        .replace(/ba detected\b/g, " ")
        .replace(/bthe detected\b/g, " ")
        .replace(/bcross domain redirects?\b/g, "redirect")
        .replace(/bcross domain flows?\b/g, "redirect")
        .replace(/bredirect chains?\b/g, "redirect")
        .replace(/bredirects? detected\b/g, "redirect")
        .replace(/bunexpected redirects?\b/g, "redirect")
        .replace(/bsuspicious redirects?\b/g, "redirect")
        .replace(/\s+/g, " ")
        .trim();
}

function getFindingDedupeKey(finding: Finding) {
    const combinedText = [
        finding.id ?? "",
        finding.message.en,
        finding.impact?.en ?? "",
        finding.remediation?.en ?? "",
    ];

```

```

].join(" ");
const normalizedText = normalizeFindingTitle(combinedText);

if (
  /\bredirect\b/.test(normalizedText) ||
  /\bcross domain\b/.test(normalizedText)
) {
  return "root:redirect";
}

if (normalizedText.includes("content security policy") || normalizedText.includes("csp")) {
  return "root:content-security-policy";
}

if (normalizedText.includes("strict transport security") || normalizedText.includes("hsts")) {
  return "root:strict-transport-security";
}

if (normalizedText.includes("x frame options") || normalizedText.includes("clickjacking")) {
  return "root:x-frame-options";
}

if (
  normalizedText.includes("x content type options") ||
  normalizedText.includes("mime sniffing")
) {
  return "root:x-content-type-options";
}

if (normalizedText.includes("referrer policy")) {
  return "root:referrer-policy";
}

if (normalizedText.includes("permissions policy")) {
  return "root:permissions-policy";
}

if (normalizedText.includes("tls") || normalizedText.includes("ssl")) {
  if (normalizedText.includes("self signed")) {
    return "root:self-signed-tls";
  }

  if (normalizedText.includes("expired") || normalizedText.includes("expires")) {
    return "root:tls-expiration";
  }

  if (normalizedText.includes("weak") || normalizedText.includes("deprecated")) {
    return "root:weak-tls";
  }

  return "root:tls";
}

if (
  normalizedText.includes("infrastructure") ||
  normalizedText.includes("server metadata")
) {
  return "root:infrastructure-exposure";
}

return `root:${normalizedText}`;
}

function getSeverityRank(severity: Severity) {
  const ranks: Record<Severity, number> = {
    critical: 5,
    high: 4,
    medium: 3,
    low: 2,
    informational: 1,
  };
  return ranks[severity];
}

function getFindingDetailScore(finding: Finding) {
  return (
    (finding.id ? 100 : 0) +
    (finding.impact ? 80 : 0) +
    (finding.remediation ? 80 : 0) +
    finding.message.en.length +
    finding.message.ar.length +
    (finding.impact?.en.length ?? 0) +
    (finding.impact?.ar.length ?? 0) +
    (finding.remediation?.en.length ?? 0) +
    (finding.remediation?.ar.length ?? 0)
  );
}

function isActionableFinding(finding: Finding) {
  const actionablePrefix = /\^(enable|add|enforce|configure|review|restrict|remove|harden|investigate|verify|reduce)\b/i;
  return (
    actionablePrefix.test(finding.message.en.trim()) ||
    actionablePrefix.test(finding.remediation?.en.trim() ?? "")
  );
}

function dedupeFindings(findings: Finding[]) {
  const uniqueFindings = new Map<string, Finding>();

  findings.forEach((finding) => {
    const key = getFindingDedupeKey(finding);
    const existing = uniqueFindings.get(key);

    if (!existing) {
      uniqueFindings.set(key, finding);
      return;
    }

    const findingDetail = getFindingDetailScore(finding);
    const existingDetail = getFindingDetailScore(existing);
    const findingSeverity = getSeverityRank(finding.severity);
    const existingSeverity = getSeverityRank(existing.severity);
    const findingActionable = isActionableFinding(finding);
    const existingActionable = isActionableFinding(existing);

    if (

```

```

        findingSeverity > existingSeverity ||
        (findingSeverity === existingSeverity &&
         findingActionable &&
         !existingActionable) ||
        (findingSeverity === existingSeverity &&
         findingActionable === existingActionable &&
         findingDetail > existingDetail)
    ) {
        uniqueFindings.set(key, finding);
    }
});
return Array.from(uniqueFindings.values());
}

function getRecommendationRootKeys(result: ScanResult) {
    const recommendationRoots = new Set<string>();

    if (result.redirects.suspicious) {
        recommendationRoots.add("root:redirect");
    }

    if (result.technologies.length > 0 || result.meta.server) {
        recommendationRoots.add("root:infrastructure-exposure");
    }

    return recommendationRoots;
}

function removeFindingRecommendationOverlaps(result: ScanResult) {
    const recommendationRoots = getRecommendationRootKeys(result);

    if (recommendationRoots.size === 0) {
        return result.findings;
    }

    return result.findings.filter((finding) => {
        const key = getFindingDedupeKey(finding);

        if (!recommendationRoots.has(key)) {
            return true;
        }

        return isActionableFinding(finding);
    });
}

function getSeverityScoreCeiling(findings: Finding[]) {
    if (findings.some((finding) => finding.severity === "critical")) {
        return 65;
    }

    if (findings.some((finding) => finding.severity === "high")) {
        return 79;
    }

    if (findings.some((finding) => finding.severity === "medium")) {
        return 89;
    }

    if (findings.some((finding) => finding.severity === "low")) {
        return 95;
    }

    return 100;
}

function getPresentHeaderCount(headers: ScanResult["headers"]) {
    return SECURITY_HEADERS.reduce(
        (total, header) => total + (headers[header] ? 1 : 0),
        0,
    );
}

function getHeaderCoverageScoreCeiling(result: ScanResult) {
    const presentHeaderCount = getPresentHeaderCount(result.headers);

    if (canReachPerfectScore(result)) {
        return 100;
    }

    if (headerObservationTelemetryIncomplete(result)) {
        const trustedSurface =
            result.intelligence.reputation === "trusted" || result.reputation?.verdict === "clean";

        return trustedSurface ? 92 : 88;
    }

    if (presentHeaderCount === 0) {
        return 45;
    }

    if (presentHeaderCount <= 2) {
        return 65;
    }

    if (presentHeaderCount <= 4) {
        return 80;
    }

    return 95;
}

function hasWafDetected(result: ScanResult) {
    return Boolean(
        result.infrastructure.waf ||
        result.infrastructure.detections.some((detection) => detection.category === "waf"),
    );
}

function isHttpsEnabled(result: ScanResult) {
    return result.meta.finalUrl.toLowerCase().startsWith("https://");
}

function hasIncompleteScanState(result: ScanResult) {
    return Object.entries(result.meta.stages).some(([stageName, stage]) => {
        if (stageName === "aiSummary") {
            return false;
        }
    });
}

```

```

    return stage.status === "timeout" || stage.status === "partial" || stage.status === "failed";
  });
}

function hasAntiBotBlocking(result: ScanResult): boolean {
  const reason = String(result.meta.stages.headers.reason ?? "");

  return (
    result.meta.statusCode === 403 ||
    reason.includes("unable_to_verify") ||
    reason.includes("anti_bot") ||
    reason.includes("challenge")
  );
}

function hasInfrastructureExposureFinding(result: ScanResult) {
  return result.findings.some(
    (finding) =>
      finding.id === "infrastructure-exposed" ||
      normalizeFindingTitle(finding.message.en).includes("infrastructure and server metadata"),
  );
}

function areAllSecurityHeadersPresent(result: ScanResult) {
  return getPresentHeaderCount(result.headers) === SECURITY_HEADERS.length;
}

function canReachPerfectScore(result: ScanResult) {
  return (
    isHttpsEnabled(result) &&
    result.ssl.valid &&
    isTls12OrNewer(result.ssl.protocol) &&
    areAllSecurityHeadersPresent(result) &&
    result.findings.length === 0 &&
    !hasIncompleteScanState(result) &&
    !hasAntiBotBlocking(result)
  );
}

function getSecurityPostureScoreCeiling(result: ScanResult) {
  const ceilings = [
    getHeaderCoverageScoreCeiling(result),
    isHttpsEnabled(result) ? 100 : 55,
    getSeverityScoreCeiling(result.findings),
  ];

  if (canReachPerfectScore(result)) {
    ceilings.push(100);
  }

  return Math.min(...ceilings);
}

function clampFinalScore(score: number) {
  return Math.max(10, Math.min(100, Math.round(score)));
}

function calculateDeterministicScore(result: ScanResult) {
  const tlsCompleted = result.meta.stages.tls.status === "completed";
  const infrastructureCompleted =
    result.meta.stages.infrastructure.status === "completed";
  const headerTelemetryIncomplete = headerObservationTelemetryIncomplete(result);
  const missingHeaders = Object.entries(result.headers)
    .filter(([_, present]) => !present)
    .map(([header]) => header);
  const missingHeaderPenalty = shouldPenalizeObservableMissingHeaders(result)
    ? missingHeaders.reduce((total, header) => {
        if (header === "content-security-policy" || header === "strict-transport-security") {
          return total + 12;
        }
      }, 0)
    : 0;
  const httpsPenalty = isHttpsEnabled(result) ? 0 : 25;
  const wafPenalty =
    hasWafDetected(result) || headerTelemetryIncomplete ? 0 : 5;
  const tlsVersionPenalty =
    isHttpsEnabled(result) && result.ssl.protocol && !isTls12OrNewer(result.ssl.protocol)
      ? 15
      : 0;
  const visibilityPenalty =
    isScannerVisibilityLimited(result) ||
    hasIncompleteScanState(result) ||
    hasAntiBotBlocking(result)
      ? 5
      : 0;
  const infrastructureExposurePenalty = hasInfrastructureExposureFinding(result) ? 3 : 0;
  const findingPenalty = result.findings.reduce(
    (total, finding) => total + FINDING_SCORE_DEDUCTION[finding.severity],
    0,
  );
  const intelligencePenalty =
    (result.intelligence.reputation === "suspicious"
      ? SCORE_WEIGHTS.suspiciousReputation
      : 0) +
    (result.intelligence.suspiciousTld ? SCORE_WEIGHTS.suspiciousTld : 0) +
    (result.intelligence.punycode ? SCORE_WEIGHTS.punycode : 0) +
    (result.intelligence.typosquatting ? SCORE_WEIGHTS.typosquatting : 0) +
    (result.intelligence.activePhishingIndicators
      ? SCORE_WEIGHTS.activePhishing
      : 0) +
    (result.intelligence.phishingKeywords.length > 0
      ? SCORE_WEIGHTS.phishingKeyword
      : 0) +
    (result.reputation?.verdict === "malicious"
      ? SCORE_WEIGHTS.reputationMalicious
      : 0) +
    (result.reputation?.verdict === "suspicious"
      ? SCORE_WEIGHTS.reputationSuspicious
      : 0) +
    (result.intelligence.excessiveSubdomains
      ? SCORE_WEIGHTS.excessiveSubdomains
      : 0) +
    (result.intelligence.entropy >= 4.2 ? SCORE_WEIGHTS.highEntropy : 0);
  const tlsPenalty = tlsCompleted
    ? (!result.ssl.valid ? SCORE_WEIGHTS.tlsInvalid : 0) +
      (result.ssl.selfSigned ? SCORE_WEIGHTS.tlsSelfSigned : 0) +

```

```

    (result.ssl.daysLeft <= 0 && result.meta.finalUrl.startsWith("https://"))
    ? SCORE_WEIGHTS.tlsExpired
    : 0) +
    (result.ssl.daysLeft > 0 && result.ssl.daysLeft <= 30
    ? SCORE_WEIGHTS.tlsExpiringSoon
    : 0) +
    (result.ssl.weakProtocol ? SCORE_WEIGHTS.tlsWeakProtocol : 0) +
    (result.ssl.weakCipher ? SCORE_WEIGHTS.tlsWeakCipher : 0)
    : 0;
const redirectPenalty = result.redirects.suspicious
    ? SCORE_WEIGHTS.redirectSuspicious
    : 0;
const statusPenalty =
    result.meta.statusCode >= 500 ? SCORE_WEIGHTS.serverError : 0;
const infrastructurePenalty =
    infrastructureCompleted && result.infrastructure.serverExposureScore >= 70
    ? SCORE_WEIGHTS.serverExposure
    : 0;
const totalPenalty =
    httpsPenalty +
    missingHeaderPenalty +
    wafPenalty +
    tlsVersionPenalty +
    visibilityPenalty +
    infrastructureExposurePenalty +
    findingPenalty +
    intelligencePenalty +
    tlsPenalty +
    redirectPenalty +
    statusPenalty +
    infrastructurePenalty;
const positiveTrustBonus = calculatePositiveTrustBonus(result);
const rawScore = 100 - totalPenalty + positiveTrustBonus;

const calibratedScore = applyScoreSanityProtection(rawScore, result);
const scoreCeiling = getSecurityPostureScoreCeiling(result);

return clampFinalScore(Math.min(calibratedScore, scoreCeiling));
}

function finalizeScore(result: ScanResult) {
    result.findings = dedupeFindings(result.findings);
    result.score = calculateDeterministicScore(result);
    result.grade = calculateGrade(result.score);
    result.threatLevel = calculateThreatLevel(result.score);
    result.confidence = calculateConfidence(result);
    result.findings = removeFindingRecommendationOverlaps(result);
    result.deterministicHash = generateDeterministicHash(result);

    return Object.freeze(result);
}

function sanitizeServerHeader(serverHeader: string | null) {
    if (!serverHeader) {
        return "";
    }

    return serverHeader
        .split(/\s+/)
        .map((part) => part.replace(/\[/\w.+-]/g, ""))
        .join(" ")
        .trim();
}

function getRegistrableDomain(hostname: string) {
    const parts = hostname.toLowerCase().split(".").filter(Boolean);

    if (parts.length <= 2) {
        return parts.join(".");
    }

    return parts.slice(-2).join(".");
}

function calculateEntropy(value: string) {
    if (!value) {
        return 0;
    }

    const counts = Array.from(value).reduce<Record<string, number>>(<
        (accumulator, char) => {
            accumulator[char] = (accumulator[char] ?? 0) + 1;
            return accumulator;
        },
        {});
    const entropy = Object.values(counts).reduce((total, count) => {
        const probability = count / value.length;
        return total - probability * Math.log2(probability);
    }, 0);

    return Number(entropy.toFixed(2));
}

function normalizeForTyposquatting(value: string) {
    return Array.from(value)
        .map((char) => TYPO_REPLACEMENTS[char] ?? char)
        .join("")
        .replace(/^[a-z]/g, "");
}

function hasTyposquattingIndicator(label: string, domain: string) {
    const normalized = normalizeForTyposquatting(label);

    return BRAND_KEYWORDS.some((brand) => {
        if (domain === `${brand}.com` || label === brand) {
            return false;
        }

        return (
            normalized.includes(brand) ||
            (Math.abs(normalized.length - brand.length) <= 2 &&
                normalized[0] === brand[0] &&
                normalized.at(-1) === brand.at(-1))
        );
    });
}

function safeDecode(value: string) {

```

```

    try {
        return decodeURIComponent(value);
    } catch {
        return value;
    }
}

function analyzeDomainIntelligence(targetUrl: URL, findings: Finding[]) {
    const hostname = targetUrl.hostname.toLowerCase();
    const domain = getRegistrableDomain(hostname);
    const labels = hostname.split(".").filter(Boolean);
    const tld = labels.at(-1) ?? "";
    const primaryLabel = domain.split(".").at(0) ?? "";
    const searchableUrl = safeDecode(targetUrl.toString()).toLowerCase();
    const phishingKeywords = PHISHING_KEYWORDS.filter((keyword) =>
        searchableUrl.includes(keyword),
    );
    const activePhishingIndicators =
        (hostname.includes("xn--") || hasTyposquattingIndicator(primaryLabel, domain)) &&
        phishingKeywords.some((keyword) =>
            ["login", "signin", "password", "verify", "account", "wallet"].includes(keyword),
        );
    const intelligence: ScanResult["intelligence"] = {
        domain,
        reputation: TRUSTED_DOMAINS.has(domain) ? "trusted" : "neutral",
        suspiciousTld: SUSPICIOUS_TLDS.has(tld),
        punycode: hostname.includes("xn--"),
        typosquatting: hasTyposquattingIndicator(primaryLabel, domain),
        phishingKeywords,
        excessiveSubdomains: labels.length > 4,
        entropy: calculateEntropy(primaryLabel),
        dnsRisk: activePhishingIndicators
            ? "high"
            : hostname.includes("xn--") || SUSPICIOUS_TLDS.has(tld)
                ? "medium"
                : "low",
        activePhishingIndicators,
    };
    if (
        intelligence.suspiciousTld ||
        intelligence.punycode ||
        intelligence.typosquatting ||
        intelligence.phishingKeywords.length > 0 ||
        intelligence.excessiveSubdomains ||
        intelligence.entropy >= 4.2
    ) {
        intelligence.reputation = "suspicious";
    }
    if (intelligence.suspiciousTld) {
        addFinding(
            findings,
            "medium",
            "The domain uses a top-level domain commonly associated with abuse.",
            "????? ???? ???? ???? ???? ???? ???? ???? ???? ????.",
        );
    }
    if (intelligence.activePhishingIndicators) {
        addFinding(
            findings,
            "critical",
            "The URL combines deceptive Domain indicators with authentication-oriented Phishing keywords.",
            "???? URL ??? ????? Domain ????? ????? Phishing ????? ????? ? ? ??????",
        );
    }
    if (intelligence.punycode) {
        addFinding(
            findings,
            "medium",
            "The domain uses punycode/IDN encoding, which can hide lookalike characters.",
            "????? ????? ???? Punycode/IDN ??? ? ???? ????? ????.",
        );
    }
    if (intelligence.typosquatting) {
        addFinding(
            findings,
            "high",
            "The domain contains typosquatting indicators related to a known brand.",
            "????? ????? ? ? ???? ???? ???? ???? ???? ???? ????.",
        );
    }
    if (intelligence.phishingKeywords.length > 0) {
        addFinding(
            findings,
            "medium",
            "The URL contains keywords frequently used in phishing campaigns.",
            "????? ????? ? ? ???? ???? ???? ? ? ???? ????.",
        );
    }
    if (intelligence.excessiveSubdomains) {
        addFinding(
            findings,
            "low",
            "The hostname uses an unusually deep subdomain structure.",
            "????? ? ? ???? ? ? ???? ???? ???? ???? ? ? ????.",
        );
    }
    if (intelligence.entropy >= 4.2) {
        addFinding(
            findings,
            "medium",
            "The primary domain label has high entropy and may be automatically generated.",
            "????? ? ? ???? ???? ???? ???? ? ? ? ? ? ? ? ? ? ?.",
        );
    }
    return intelligence;
}

function getIssuerName(certificate: PeerCertificate) {
    return (
        certificate.issuer?.0 ||
    )
}

```

```

        certificate.issuer?.CN ||
        certificate.issuer?.OU ||
        "Unknown issuer"
    );
}

function getCertificateDaysLeft(validTo?: string) {
    if (!validTo) {
        return 0;
    }

    const expiresAt = new Date(validTo).getTime();

    if (Number.isNaN(expiresAt)) {
        return 0;
    }

    return Math.ceil((expiresAt - Date.now()) / 86_400_000);
}

function isSelfSignedCertificate(socket: TlsSocketLike, certificate: PeerCertificate) {
    const issuerCertificate = certificate.issuerCertificate;
    const sameFingerprint =
        Boolean(issuerCertificate?.fingerprint256) &&
        issuerCertificate?.fingerprint256 === certificate.fingerprint256;
    const sameSubjectAndIssuer =
        certificate.subject?.CN !== undefined &&
        certificate.subject.CN === certificate.issuer?.CN;

    return (
        socket.authorizationError === "DEPTH_ZERO_SELF_SIGNED_CERT" ||
        sameFingerprint ||
        sameSubjectAndIssuer
    );
}

function isWeakTlsProtocol(protocol: string) {
    return ["SSLv2", "SSLv3", "TLSv1", "TLSv1.1"].includes(protocol);
}

function isWeakCipher(cipher: string) {
    const normalized = cipher.toUpperCase();

    return (
        normalized.includes("RC4") ||
        normalized.includes("3DES") ||
        normalized.includes("DES") ||
        normalized.includes("MD5") ||
        normalized.includes("NULL") ||
        normalized.includes("EXPORT")
    );
}

function isRedirectStatus(statusCode: number) {
    return [301, 302, 303, 307, 308].includes(statusCode);
}

function isAllowedContentType(contentType: string | null) {
    if (!contentType) {
        return true;
    }

    const normalized = contentType.split(";")[0]?.trim().toLowerCase();

    return normalized === "text/html" || normalized === "application/xhtml+xml";
}

function isResponseTooLarge(contentLength: string | null) {
    if (!contentLength) {
        return false;
    }

    const size = Number(contentLength);

    return Number.isFinite(size) && size > MAX_RESPONSE_BYTES;
}

function isAntiBotOrChallengeResponse(response: Response) {
    const headers = Array.from(response.headers.entries())
        .map(([key, value]) => `${key}:${value}`)
        .join(" ")
        .toLowerCase();
    const server = response.headers.get("server")?.toLowerCase() ?? "";

    return (
        response.status === 403 ||
        headers.includes("cf-mitigated") ||
        headers.includes("cf-chl") ||
        headers.includes("cloudflare challenge") ||
        headers.includes("akamai bot") ||
        headers.includes("x-sucuri-block") ||
        headers.includes("datadome") ||
        headers.includes("perimeterx") ||
        server.includes("cloudflare")
    );
}

/**
 * Many providers strip or reshape headers for automated clients. A small response with none of the
 * tracked security headers is not proof those controls are absent site-wide ? only that we could
 * not observe them on this surface.
 */
function shouldTreatFinalResponseAsLimitedObservableSurface(
    response: Response,
    headersUnableToVerify: boolean,
): boolean {
    if (headersUnableToVerify) {
        return true;
    }

    if (response.status === 403 || response.status === 503) {
        return true;
    }

    const distinctKeys = Array.from(response.headers.keys()).length;
    const anyTrackedSecurityHeaderPresent = SECURITY_HEADERS.some((header) =>
        response.headers.has(header),
    );
}

```



```

    return distinctKeys <= 14 && !anyTrackedSecurityHeaderPresent;
}

/**
 * Header-specific observability only (TLS/redirect partial stages alone must not waive posture caps).
 */
function headerObservationTelemetryIncomplete(result: ScanResult): boolean {
    const reason = String(result.meta.stages.headers.reason ?? "").toLowerCase();

    return (
        result.meta.stages.headers.status !== "completed" ||
        reason.includes("unable_to_verify") ||
        reason.includes("limited_observable_surface") ||
        hasAntiBotBlocking(result)
    );
}

/** Missing-header penalties apply only when we actually observed a normal HTML surface end-to-end. */
function shouldPenalizeObservableMissingHeaders(result: ScanResult): boolean {
    if (result.meta.stages.headers.status !== "completed") {
        return false;
    }

    const reason = String(result.meta.stages.headers.reason ?? "").toLowerCase();

    if (reason.includes("unable_to_verify") || reason.includes("limited_observable_surface")) {
        return false;
    }

    return !hasAntiBotBlocking(result);
}

async function inspectTlsCertificate(
    targetUrl: URL,
    signal: AbortSignal,
    timeoutMs: number,
) {
    if (targetUrl.protocol !== "https:") {
        return {
            valid: false,
            selfSigned: false,
            issuer: "",
            daysLeft: 0,
            protocol: "",
            cipher: "",
            weakProtocol: false,
            weakCipher: false,
        };
    }

    assertNodeRuntime();

    const { connect } = await loadTls();

    return new Promise<ScanResult["ssl"]>((resolve) => {
        const socket = connect({
            host: targetUrl.hostname,
            port: Number(targetUrl.port || 443),
            servername: targetUrl.hostname,
            rejectUnauthorized: false,
        }) as TlsSocketLike;

        const cleanup = () => {
            clearTimeout(timer);
            signal.removeEventListener("abort", abort);
            socket.removeAllListeners();
            socket.destroy();
        };

        const finish = (ssl: ScanResult["ssl"]) => {
            cleanup();
            resolve(ssl);
        };

        const abort = () => {
            finish({
                valid: false,
                selfSigned: false,
                issuer: "",
                daysLeft: 0,
                protocol: "",
                cipher: "",
                weakProtocol: false,
                weakCipher: false,
            });
        };

        const timer = setTimeout(abort, timeoutMs);

        signal.addEventListener("abort", abort, { once: true });

        socket.once("secureConnect", () => {
            const certificate = socket.getPeerCertificate(true) as PeerCertificate;

            if (!certificate || Object.keys(certificate).length === 0) {
                finish({
                    valid: false,
                    selfSigned: false,
                    issuer: "",
                    daysLeft: 0,
                    protocol: "",
                    cipher: "",
                    weakProtocol: false,
                    weakCipher: false,
                });
                return;
            }

            const daysLeft = getCertificateDaysLeft(certificate.valid_to);
            const selfSigned = isSelfSignedCertificate(socket, certificate);
            const tlsProtocol = socket.getProtocol() ?? "";
            const cipher = socket.getCipher();
            const cipherName = cipher.standardName || cipher.name || "";

            finish({
                valid: socket.authorized && !selfSigned && daysLeft > 0,
                selfSigned,
                issuer: getIssuerName(certificate),
                daysLeft,
            });
        });
    });
}

```

```

        protocol: tlsProtocol,
        cipher: cipherName,
        weakProtocol: isWeakTlsProtocol(tlsProtocol),
        weakCipher: isWeakCipher(cipherName),
    });
});
socket.once("error", abort);
});
}

async function safeFetch(url: string, signal: AbortSignal): Promise<FetchStep> {
    const headResponse = await fetch(url, {
        method: "HEAD",
        redirect: "manual",
        cache: "no-store",
        signal,
    });

    if (![405, 501].includes(headResponse.status)) {
        return {
            response: headResponse,
            url,
            statusCode: headResponse.status,
        };
    }

    // Fallback keeps bandwidth low when HEAD is unsupported. The body is never
    // parsed or rendered, and it is cancelled as soon as response headers arrive.
    const getResponse = await fetch(url, {
        method: "GET",
        redirect: "manual",
        cache: "no-store",
        headers: {
            Accept: "text/html, application/xhtml+xml",
            Range: "bytes=0-0",
        },
        signal,
    });

    await getResponse.body?.cancel();

    return {
        response: getResponse,
        url,
        statusCode: getResponse.status,
    };
}

async function safeFetchWithRetry(
    url: string,
    parentSignal: AbortSignal,
): Promise<FetchStep> {
    return retryTransient(() =>
        withStageTimeout("headers", STAGE_TIMEOUTS.headers, async (fetchSignal) => {
            const combined = combineSignals(parentSignal, fetchSignal);

            try {
                return await safeFetch(url, combined.signal);
            } finally {
                combined.cleanup();
            }
        })
    );
}

async function validateWithDnsTimeout(
    input: string,
    validateUrl: ValidateUrl,
): Promise<ValidationResult> {
    let lastResult: ValidationResult | null = null;
    let lastError: unknown;

    for (let attempt = 1; attempt <= TRANSIENT_RETRY_ATTEMPTS; attempt += 1) {
        let result: ValidationResult;

        try {
            result = await withStageTimeout("dns", STAGE_TIMEOUTS.dns, () =>
                validateUrl(input),
            );
        } catch (error) {
            lastError = error;
        }

        if (!isTransientError(error) || attempt === TRANSIENT_RETRY_ATTEMPTS) {
            throw error;
        }

        continue;
    }

    if (
        result.valid ||
        !["dns_timeout", "dns_failed"].includes(result.blockedReason ?? "") ||
        attempt === TRANSIENT_RETRY_ATTEMPTS
    ) {
        return result;
    }

    if (lastError) {
        throw lastError;
    }

    return lastResult ?? { valid: false, url: input, blockedReason: "dns_failed" };
}

async function followRedirects(
    initialUrl: string,
    signal: AbortSignal,
    findings: Finding[],
    validateUrl: ValidateUrl,
) {
    const chain: RedirectStep[] = [];
    const visited = new Set<string>();
    const initialDomain = new URL(initialUrl).hostname.toLowerCase();
    let currentUrl = initialUrl;
    let suspicious = false;
    let finalStep: FetchStep | null = null;

```

```

for (let depth = 0; depth <= MAX_REDIRECTS; depth += 1) {
  const validation = await validateUrl(currentUrl);

  if (!validation.valid) {
    addFinding(
      findings,
      "high",
      "A redirect target failed URL safety validation.",
      "??? ??? ????? ?????? ?? ??? ????? ??????",
    );
    break;
  }

  currentUrl = validation.url;

  if (visited.has(currentUrl)) {
    suspicious = true;
    addFinding(
      findings,
      "high",
      "A redirect loop was detected.",
      "?? ?????? ??? ???? ??????",
      "redirect-loop",
    );
    break;
  }

  visited.add(currentUrl);

  const step = await safeFetchWithRetry(currentUrl, signal);
  chain.push({
    url: currentUrl,
    statusCode: step.statusCode,
  });

  if (!isRedirectStatus(step.statusCode)) {
    finalStep = step;
    break;
  }

  if (depth === MAX_REDIRECTS) {
    suspicious = true;
    addFinding(
      findings,
      "medium",
      "Redirect depth exceeded the maximum allowed limit.",
      "????? ?????? ????? ?????? ??? ?????? ?????? ??????",
      "suspicious-redirect",
    );
    break;
  }

  const location = step.response.headers.get("location");

  if (!location) {
    finalStep = step;
    break;
  }

  const nextUrl = new URL(location, currentUrl).toString();
  const nextDomain = new URL(nextUrl).hostname.toLowerCase();

  if (nextDomain !== initialDomain) {
    suspicious = true;
    addFinding(
      findings,
      "medium",
      "A cross-domain redirect was detected.",
      "?? ?????? ????? ?????? ??? ??? ??????",
      "cross-domain-redirect",
    );
  }

  currentUrl = nextUrl;
}

return {
  chain,
  suspicious,
  finalStep,
  finalUrl: finalStep?.url || currentUrl,
};
}

function analyzeSecurityHeaders(
  response: Response,
  findings: Finding[],
  unableToVerify = false,
) {
  const headers: Record<string, boolean> = {};
  const headerFindingIds: Partial<Record<(typeof SECURITY_HEADERS)[number], IntelligenceFindingId>> = {
    "strict-transport-security": "missing-hsts",
    "content-security-policy": "missing-csp",
    "x-frame-options": "missing-x-frame-options",
    "x-content-type-options": "missing-x-content-type-options",
    "referrer-policy": "missing-referrer-policy",
    "permissions-policy": "missing-permissions-policy",
  };

  for (const header of SECURITY_HEADERS) {
    const present = response.headers.has(header);
    headers[header] = present;

    if (!present && !unableToVerify) {
      addFinding(
        findings,
        "low",
        `Missing security header: ${header}.`,
        `??? ?????? ?????: ${header}.`,
        headerFindingIds[header],
      );
    }
  }

  return headers;
}

function analyzeResponseSafety(response: Response, findings: Finding[]) {

```

```

if (isResponseTooLarge(response.headers.get("content-length"))) {
    addFinding(
        findings,
        "medium",
        "Response size exceeds the scanner safety limit.",
        "??? ???????? ?????? ?? ?????? ?????? ???????"
    );
}

if (!isAllowedContentType(response.headers.get("content-type"))) {
    addFinding(
        findings,
        "medium",
        "Unsupported or non-HTML content type was detected.",
        "?? ?????? ??? ?????? ?? ?????? ?? ?? HTML."
    );
}
}

function addTechnology(technologies: Set<string>, value: string) {
    technologies.add(value);
}

function createPassiveSignals(
    response: Response,
    finalUrl: string,
    redirects: ScanResult["redirects"],
): PassiveSignals {
    const server = response.headers.get("server")?.toLowerCase() ?? "";
    const poweredBy = response.headers.get("x-powered-by")?.toLowerCase() ?? "";
    const via = response.headers.get("via")?.toLowerCase() ?? "";
    const cookies = response.headers.get("set-cookie")?.toLowerCase() ?? "";
    const headerNames = Array.from(response.headers.keys()).join(" ").toLowerCase();
    const headerValues = Array.from(response.headers.entries())
        .map(([key, value]) => `${key}:${value}`)
        .join(" ")
        .toLowerCase();
    let finalHostname = "";

    try {
        finalHostname = new URL(finalUrl).hostname.toLowerCase();
    } catch {
        finalHostname = "";
    }

    const redirectHostnames = redirects.chain
        .map((step) => {
            try {
                return new URL(step.url).hostname.toLowerCase();
            } catch {
                return "";
            }
        })
        .filter(Boolean);

    return {
        headerNames,
        headerValues,
        server,
        poweredBy,
        via,
        cookies,
        finalHostname,
        redirectHostnames,
    };
}

function hasSignal(signals: PassiveSignals, patterns: string[]) {
    return patterns.some((pattern) =>
        signals.headerNames.includes(pattern) ||
        signals.headerValues.includes(pattern) ||
        signals.server.includes(pattern) ||
        signals.poweredBy.includes(pattern) ||
        signals.via.includes(pattern) ||
        signals.cookies.includes(pattern) ||
        signals.finalHostname.includes(pattern) ||
        signals.redirectHostnames.some((hostname) => hostname.includes(pattern)),
    );
}

function addCandidate(
    candidates: Candidate[],
    category: InfrastructureDetection["category"],
    name: string,
    weight: number,
    signal: string,
) {
    const existing = candidates.find(
        (candidate) => candidate.category === category && candidate.name === name,
    );

    if (existing) {
        existing.score += weight;
        existing.signals.push(signal);
        return;
    }

    candidates.push({
        category,
        name,
        score: weight,
        signals: [signal],
    });
}

function collectPassiveInfrastructureCandidates(signals: PassiveSignals) {
    const candidates: Candidate[] = [];

    if (hasSignal(signals, ["cf-ray", "cf-cache-status", "_cf_bm", "cloudflare"])) {
        addCandidate(candidates, "cdn", "Cloudflare", 45, "cloudflare_edge_headers");
        addCandidate(candidates, "waf", "Cloudflare WAF", 35, "cloudflare_security_headers");
        addCandidate(candidates, "reverseProxy", "Cloudflare", 42, "cloudflare_reverse_proxy");
        addCandidate(candidates, "asn", "Cloudflare AS13335", 42, "cloudflare_network_signal");
        addCandidate(candidates, "ipOwner", "Cloudflare", 42, "cloudflare_network_signal");
    }

    if (hasSignal(signals, ["x-vercel-id", "x-vercel-cache", "vercel", ".vercel.app"])) {
        addCandidate(candidates, "cdn", "Vercel Edge Network", 38, "vercel_edge_headers");
        addCandidate(candidates, "hosting", "Vercel", 45, "vercel_platform_headers");
    }
}

```

```

    addCandidate(candidates, "cloud", "Vercel", 34, "vercel_platform_headers");
    addCandidate(candidates, "reverseProxy", "Vercel Edge", 40, "vercel_edge_headers");
    addCandidate(candidates, "asn", "Vercel AS398101", 40, "vercel_network_signal");
    addCandidate(candidates, "ipOwner", "Vercel", 40, "vercel_network_signal");
}

if (hasSignal(signals, ["x-nf-request-id", "netlify", ".netlify.app"])) {
    addCandidate(candidates, "cdn", "Netlify Edge", 36, "netlify_edge_headers");
    addCandidate(candidates, "hosting", "Netlify", 45, "netlify_platform_headers");
    addCandidate(candidates, "cloud", "Netlify", 30, "netlify_platform_headers");
    addCandidate(candidates, "reverseProxy", "Netlify Edge", 40, "netlify_edge_headers");
    addCandidate(candidates, "ipOwner", "Netlify", 40, "netlify_network_signal");
}

if (hasSignal(signals, ["rندر-id", "render", ".onrender.com"])) {
    addCandidate(candidates, "hosting", "Render", 45, "render_platform_headers");
    addCandidate(candidates, "cloud", "Render", 30, "render_platform_headers");
    addCandidate(candidates, "reverseProxy", "Render Edge", 40, "render_edge_signal");
    addCandidate(candidates, "ipOwner", "Render", 40, "render_network_signal");
}

if (hasSignal(signals, ["x-amz", "cloudfront", "amazon", "aws", "awselb", "elb"])) {
    addCandidate(candidates, "cdn", "AWS CloudFront", 34, "aws_edge_headers");
    addCandidate(candidates, "cloud", "AWS", 45, "aws_headers");
    addCandidate(candidates, "hosting", "AWS", 30, "aws_headers");
    addCandidate(candidates, "reverseProxy", "AWS Elastic Load Balancer", 40, "aws_load_balancer_signal");
    addCandidate(candidates, "asn", "Amazon AS16509", 42, "aws_network_signal");
    addCandidate(candidates, "ipOwner", "Amazon Web Services", 42, "aws_network_signal");
}

if (hasSignal(signals, ["azure", "x-azure", "azurewebsites.net", "microsoft-iis"])) {
    addCandidate(candidates, "cloud", "Azure", 42, "azure_headers");
    addCandidate(candidates, "hosting", "Azure App Service", 32, "azure_hosting_signal");
    addCandidate(candidates, "asn", "Microsoft AS8075", 42, "azure_network_signal");
    addCandidate(candidates, "ipOwner", "Microsoft Azure", 42, "azure_network_signal");
}

if (hasSignal(signals, ["google", "gcp", "appspot.com", "run.app", "x-cloud-trace-context"])) {
    addCandidate(candidates, "cloud", "GCP", 42, "gcp_headers");
    addCandidate(candidates, "hosting", "Google Cloud", 32, "gcp_hosting_signal");
    addCandidate(candidates, "asn", "Google AS15169", 42, "gcp_network_signal");
    addCandidate(candidates, "ipOwner", "Google Cloud", 42, "gcp_network_signal");
}

if (hasSignal(signals, ["sucuri", "x-sucuri-id", "x-sucuri-cache"])) {
    addCandidate(candidates, "waf", "Sucuri WAF", 42, "sucuri_security_headers");
    addCandidate(candidates, "cdn", "Sucuri CDN", 25, "sucuri_edge_headers");
}

if (hasSignal(signals, ["akamai", "x-akamai", "akamai-ghost"])) {
    addCandidate(candidates, "cdn", "Akamai", 42, "akamai_headers");
    addCandidate(candidates, "waf", "Akamai WAF", 40, "akamai_security_signal");
    addCandidate(candidates, "asn", "Akamai AS20940", 42, "akamai_network_signal");
    addCandidate(candidates, "ipOwner", "Akamai", 42, "akamai_network_signal");
}

if (hasSignal(signals, ["fastly", "x-served-by", "x-cache-hits"])) {
    addCandidate(candidates, "cdn", "Fastly", 40, "fastly_headers");
    addCandidate(candidates, "reverseProxy", "Fastly", 40, "fastly_proxy_signal");
    addCandidate(candidates, "asn", "Fastly AS54113", 42, "fastly_network_signal");
    addCandidate(candidates, "ipOwner", "Fastly", 42, "fastly_network_signal");
}

if (hasSignal(signals, ["nginx"])) {
    addCandidate(candidates, "reverseProxy", "nginx", 40, "nginx_server_header");
    addCandidate(candidates, "server", "nginx", 42, "nginx_server_header");
}

if (hasSignal(signals, ["apache"])) {
    addCandidate(candidates, "server", "Apache", 42, "apache_server_header");
    addCandidate(candidates, "reverseProxy", "Apache", 40, "apache_server_header");
}

if (hasSignal(signals, ["express"])) {
    addCandidate(candidates, "framework", "Express", 42, "express_powered_by_header");
}

if (hasSignal(signals, ["nextjs", "next.js", "x-nextjs", "_next", "__next", "next-router"])) {
    addCandidate(candidates, "framework", "Next.js", 42, "nextjs_headers_or_cookies");
}

if (hasSignal(signals, ["wordpress", "wp-", "x-redirect-by:wordpress"])) {
    addCandidate(candidates, "framework", "WordPress", 42, "wordpress_headers_or_cookies");
}

return candidates;
}

function toDetection(candidate: Candidate): InfrastructureDetection {
    const uniqueSignals = Array.from(new Set(candidate.signals)).sort((a, b) =>
        a.localeCompare(b),
    );
    const signalBonus = uniqueSignals.length > 1 ? 14 : 0;

    return {
        category: candidate.category,
        name: candidate.name,
        confidence: Math.min(95, candidate.score + signalBonus),
        signals: uniqueSignals,
    };
}

function selectDetection(
    detections: InfrastructureDetection[],
    category: InfrastructureDetection["category"],
) {
    return detections
        .filter((detection) => detection.category === category && detection.confidence >= 40)
        .sort((left, right) => {
            if (right.confidence !== left.confidence) {
                return right.confidence - left.confidence;
            }

            return left.name.localeCompare(right.name);
        })[0];
}

function detectTechnologies(response: Response, finalUrl: string, redirects: ScanResult["redirects"]) {
    const technologies = new Set<string>();

```

```

const signals = createPassiveSignals(response, finalUrl, redirects);
const detections = collectPassiveInfrastructureCandidates(signals).map(toDetection);

if (selectDetection(detections, "server")?.name === "nginx") {
  addTechnology(technologies, "nginx");
}

if (selectDetection(detections, "server")?.name === "Apache") {
  addTechnology(technologies, "Apache");
}

if (selectDetection(detections, "cdn")?.name === "Cloudflare") {
  addTechnology(technologies, "Cloudflare");
}

if (selectDetection(detections, "hosting")?.name === "Vercel") {
  addTechnology(technologies, "Vercel");
}

if (selectDetection(detections, "hosting")?.name === "Netlify") {
  addTechnology(technologies, "Netlify");
}

if (selectDetection(detections, "hosting")?.name === "Render") {
  addTechnology(technologies, "Render");
}

if (selectDetection(detections, "cloud")?.name === "AWS") {
  addTechnology(technologies, "AWS");
}

if (selectDetection(detections, "cloud")?.name === "Azure") {
  addTechnology(technologies, "Azure");
}

if (selectDetection(detections, "cloud")?.name === "GCP") {
  addTechnology(technologies, "GCP");
}

if (detections.some((detection) => detection.name === "Express" && detection.confidence >= 40)) {
  addTechnology(technologies, "Express");
}

if (detections.some((detection) => detection.name === "Next.js" && detection.confidence >= 40)) {
  addTechnology(technologies, "Next.js");
}

if (detections.some((detection) => detection.name === "WordPress" && detection.confidence >= 40)) {
  addTechnology(technologies, "WordPress");
}

if (hasSignal(signals, ["varnish", "x-varnish"])) {
  addTechnology(technologies, "Varnish");
}

return Array.from(technologies).sort((a, b) => a.localeCompare(b));
}

function analyzeInfrastructure(
  response: Response,
  technologies: string[],
  redirects: ScanResult["redirects"],
  finalUrl: string,
) {
  const signals = createPassiveSignals(response, finalUrl, redirects);
  const detections = collectPassiveInfrastructureCandidates(signals)
    .map(toDetection)
    .filter((detection) => detection.confidence >= 35)
    .sort((left, right) => {
      if (left.category !== right.category) {
        return left.category.localeCompare(right.category);
      }

      if (right.confidence !== left.confidence) {
        return right.confidence - left.confidence;
      }

      return left.name.localeCompare(right.name);
    });

  const cdn = selectDetection(detections, "cdn")?.name ?? "";
  const waf = selectDetection(detections, "waf")?.name ?? "";
  const hostingProvider = selectDetection(detections, "hosting")?.name ?? "";
  const cloudProvider = selectDetection(detections, "cloud")?.name ?? "";
  const asn = selectDetection(detections, "asn")?.name ?? "";
  const reverseProxy = selectDetection(detections, "reverseProxy")?.name ?? "";
  const framework = selectDetection(detections, "framework")?.name ?? "";
  const ipOwner = selectDetection(detections, "ipOwner")?.name ?? "";
  const confidence =
    detections.length > 0
      ? Math.round(
          detections.reduce((total, detection) => total + detection.confidence, 0) /
            detections.length,
        )
      : 0;

  const server = response.headers.get("server")?.toLowerCase() ?? "";
  const exposedServerTokens =
    (server ? 1 : 0) +
    (signals.poweredBy ? 1 : 0) +
    (framework ? 1 : 0) +
    (reverseProxy && !cdn ? 1 : 0);
  const serverExposureScore = Math.min(
    100,
    exposedServerTokens * 18 +
    technologies.length * 8 +
    (signals.cookies ? 8 : 0) +
    (waf ? -10 : 0) +
    (cdn ? -8 : 0) +
    (confidence >= 75 ? -6 : 0),
  );

  return {
    cdn,
    waf,
    hostingProvider,
    cloudProvider,
    asn,
    reverseProxy,
    framework,
    ipOwner,
  };
}

```

```

confidence,
detections,
serverExposureScore: Math.max(0, serverExposureScore),
redirectTrust: redirects.suspicious
? "suspicious"
: cdn || waf || hostingProvider || cloudProvider
? "trusted"
: "neutral",
} satisfies ScanResult["infrastructure"];
}

function analyzeSslFindings(
  ssl: ScanResult["ssl"],
  protocol: string,
  findings: Finding[],
) {
  if (protocol !== "https:") {
    addFinding(
      findings,
      "medium",
      "The URL does not use HTTPS, so no TLS certificate is available.",
      "?????? ?? ?????? HTTPS? ??? ? ???? ???? TLS.",
    );
    return;
  }

  if (!ssl.issuer) {
    addFinding(
      findings,
      "high",
      "No TLS certificate could be detected.",
      "??? ?????? TLS.",
    );
  }

  if (ssl.selfSigned) {
    addFinding(
      findings,
      "high",
      "The TLS certificate appears to be self-signed.",
      "???? ???? TLS ???? ????.",
      "self-signed-ssl",
    );
  }

  if (ssl.daysLeft <= 0) {
    addFinding(
      findings,
      "high",
      "The TLS certificate is expired.",
      "???? ???? ???? TLS.",
      "expired-ssl",
    );
  } else if (ssl.daysLeft <= 30) {
    addFinding(
      findings,
      "medium",
      "The TLS certificate expires within 30 days.",
      "???? ???? ???? TLS ??? 30 ????.",
      "ssl-expiring-soon",
    );
  }

  if (ssl.weakProtocol) {
    addFinding(
      findings,
      "high",
      "The server negotiated a deprecated TLS protocol.",
      "???? ???? ???? ???? ???? TLS ???.",
      "weak-tls",
    );
  }

  if (ssl.weakCipher) {
    addFinding(
      findings,
      "medium",
      "The negotiated TLS cipher appears weak or deprecated.",
      "???? ???? ???? ???? ???? ???? ? ? ????.",
      "weak-tls",
    );
  }
}

export async function scanUrl(input: string): Promise<ScanResult> {
  const startedAt = Date.now();
  const result = createEmptyResult();
  result.meta.scanTimestamp = new Date(startedAt).toISOString();

  try {
    assertNodeRuntime();

    const validateUrl = await loadValidator();
    const dnsStartedAt = Date.now();
    let validation: ValidationResult;

    try {
      validation = await validateWithDnsTimeout(input, validateUrl);
      updateStage(
        result,
        "dns",
        validation.valid ? "completed" : "failed",
        dnsStartedAt,
        validation.valid ? undefined : validation.blockedReason ?? "dns_failed",
      );
    } catch (error) {
      const reason = getErrorReason(error, "dns_failed");
      updateStage(
        result,
        "dns",
        reason.includes("timeout") ? "timeout" : "failed",
        dnsStartedAt,
        reason,
      );
      addFinding(
        result.findings,
        "informational",
        `DNS validation could not complete reliably (${reason}).`,
        `??? ???? ???? DNS ???? (${reason}).`,
      );
    }
  }
}

```

```

    );
    result.meta.finalUrl = input;
    result.meta.responseTime = Date.now() - startedAt;
    return finalizeScore(result);
}

result.meta.finalUrl = validation.url;

if (!validation.valid) {
    addFinding(
        result.findings,
        "high",
        "URL validation failed before scanning.",
        "??? ?????? ?? ?????? ??? ??? ??????",
    );
    return finalizeScore(result);
}

const targetUrl = new URL(validation.url);
result.intelligence = analyzeDomainIntelligence(targetUrl, result.findings);
result.reputation = await checkDomainReputation(result.intelligence.domain);

const tlsStartedAt = Date.now();

try {
    result.ssl = await retryTransient(() =>
        withStageTimeout("tls", STAGE_TIMEOUTS.tls, (signal) =>
            inspectTlsCertificate(targetUrl, signal, STAGE_TIMEOUTS.tls),
        ),
    );
    updateStage(result, "tls", "completed", tlsStartedAt);
    analyzeSslFindings(result.ssl, targetUrl.protocol, result.findings);
} catch (error) {
    const reason = getErrorReason(error, "tls_failed");
    updateStage(
        result,
        "tls",
        reason.includes("timeout") ? "timeout" : "partial",
        tlsStartedAt,
        reason,
    );
    addFinding(
        result.findings,
        "informational",
        `TLS inspection was incomplete (${reason}); confidence was reduced instead of treating the certifi
cate as failed.`,
        `?? ?????? ??? TLS (${reason})? ?? ??? ?????? ?????? ?? ?????? ?????? ??????`,
    );
}

const redirectsStartedAt = Date.now();
let redirectResult: Awaited<ReturnType<typeof followRedirects>>;

try {
    redirectResult = await withStageTimeout(
        "redirects",
        STAGE_TIMEOUTS.redirects,
        (signal) =>
            followRedirects(validation.url, signal, result.findings, validateUrl),
    );
    updateStage(result, "redirects", "completed", redirectsStartedAt);
} catch (error) {
    const reason = getErrorReason(error, "redirects_failed");
    updateStage(
        result,
        "redirects",
        reason.includes("timeout") ? "timeout" : "partial",
        redirectsStartedAt,
        reason,
    );

    if (reason.includes("headers_timeout")) {
        updateStage(result, "headers", "timeout", redirectsStartedAt, reason);
    }

    addFinding(
        result.findings,
        "informational",
        `HTTP/redirect inspection was incomplete (${reason}); the report contains partial externally obser
vable intelligence.`,
        `?? ?????? ??? HTTP/Redirects (${reason})? ?????? ?????? ??? ?????? ?????? ??????`,
    );
    result.meta.responseTime = Date.now() - startedAt;
    return finalizeScore(result);
}

result.redirects = {
    chain: redirectResult.chain,
    suspicious: redirectResult.suspicious,
};
result.meta.finalUrl = redirectResult.finalUrl;

if (!redirectResult.finalStep) {
    updateStage(
        result,
        "redirects",
        redirectResult.chain.length > 0 ? "partial" : "failed",
        redirectsStartedAt,
        "final_response_unavailable",
    );
    addFinding(
        result.findings,
        "medium",
        "The scan stopped before a final response was reached.",
        "???? ?????? ??? ?????? ??? ?????? ??????",
    );
    return finalizeScore(result);
}

const finalResponse = redirectResult.finalStep.response;
const headersStartedAt = Date.now();
const headersUnableToVerify = isAntiBotOrChallengeResponse(finalResponse);
const limitedObservableSurface = shouldTreatFinalResponseAsLimitedObservableSurface(
    finalResponse,
    headersUnableToVerify,
);

result.headers = analyzeSecurityHeaders(
    finalResponse,

```



```

        result.findings,
        limitedObservableSurface,
    );
    analyzeResponseSafety(finalResponse, result.findings);

    const headersStageStatus: ScanStageState["status"] = limitedObservableSurface
        ? "partial"
        : "completed";
    const headersStageReason = headersUnableToVerify
        ? "unable_to_verify_antibot"
        : limitedObservableSurface
            ? "limited_observable_surface"
            : undefined;

    updateStage(result, "headers", headersStageStatus, headersStartedAt, headersStageReason);

    const infrastructureStartedAt = Date.now();

    try {
        await withStageTimeout("infrastructure", STAGE_TIMEOUTS.infrastructure, async () => {
            result.technologies = detectTechnologies(
                finalResponse,
                redirectResult.finalUrl,
                result.redirects,
            );
            result.infrastructure = analyzeInfrastructure(
                finalResponse,
                result.technologies,
                result.redirects,
                redirectResult.finalUrl,
            );

            if (result.infrastructure.serverExposureScore >= 40) {
                addFinding(
                    result.findings,
                    "low",
                    "Infrastructure and server metadata are externally visible.",
                    "?????? Infrastructure ?Server ????? ???????.",
                    "infrastructure-exposed",
                );
            }
        });
        updateStage(result, "infrastructure", "completed", infrastructureStartedAt);
    } catch (error) {
        const reason = getErrorReason(error, "infrastructure_failed");
        updateStage(
            result,
            "infrastructure",
            reason.includes("timeout") ? "timeout" : "partial",
            infrastructureStartedAt,
            reason,
        );
        addFinding(
            result.findings,
            "informational",
            "Infrastructure fingerprinting was incomplete (${reason}).",
            "?? ????? ??? Infrastructure (${reason}).",
        );
    }

    result.meta = {
        ...result.meta,
        responseTime: Date.now() - startedAt,
        statusCode: finalResponse.status,
        finalUrl: redirectResult.finalUrl,
        server: sanitizeServerHeader(finalResponse.headers.get("server")),
        scanTimestamp: new Date(startedAt).toISOString(),
    };

    if (finalResponse.status >= 500) {
        addFinding(
            result.findings,
            "medium",
            "The server returned an error status code.",
            "???? ?????? ??? ????? ????? ??? ???.",
        );
    }

    return finalizeScore(result);
} catch (error) {
    const reason = getErrorReason(error, "scan_failed");

    addFinding(
        result.findings,
        reason.includes("timeout") ? "informational" : "medium",
        "The scan completed with partial intelligence because a stage failed (${reason}).",
        "????? ?????? ???????? ?????? ??? ??? ?????? (${reason}).",
    );

    result.meta.responseTime = Date.now() - startedAt;

    return finalizeScore(result);
}
}

```